

Python 语法手册

—— 配 ServerTimeDemo 项目实例

从词法到工程约定，逐条对照真实代码

74 个 Python 文件 · 44,314 行 · 23 张数据表

项目版本 1.9.0

手册版本 1.0

生成日期 2026 年 09 月 21 日

本手册的每一段示例代码都摘自本项目源码，未作伪写

目录

第一章 手册说明与项目全景	6
1.1 怎么用这本手册	6
1.2 这个项目是什么	6
1.3 代码规模	7
1.4 分层与数据流	7
1.5 全模块地图	8
1.6 阅读约定	11
第二章 词法与基础语法	12
2.1 文件头：编码声明与模块 docstring	12
2.2 注释：为什么这么写	12
2.3 命名规范	13
2.4 缩进与续行	14
2.5 运算符速查	14
2.6 为什么这里必须用字节而不是字符	15
第三章 变量与内置类型	17
3.1 赋值与解包的各种形态	17
3.2 数字：int / float / bool	17
3.3 字符串	18
3.4 真值判定与 None	19
3.5 类型转换	20
第四章 容器类型	22
4.1 list：本项目的界面行就是它	22
4.2 tuple：固定结构与「可哈希」	23
4.3 dict：本项目的配置与行数据主力	23
4.4 set：判断「有没有」比 list 快得多	24
4.5 推导式：一行构造容器	25
4.6 排序与 key	26
4.7 可变与不可变：一个必须记住的陷阱	27
第五章 控制流	28

5.1 if / elif / else 与条件表达式	28
5.2 守卫子句：把嵌套压平	28
5.3 for 循环与它的好搭档	29
5.4 while 与 break / continue	30
5.5 海象运算符 := (3.8+)	31
5.6 match / case (3.10+)	31
第六章 函数	33
6.1 定义、返回值、多返回值	33
6.2 参数的四种形态	33
6.3 可变默认参数的陷阱（第 4.7 节的正脸）	34
6.4 作用域：LEGB 与闭包	34
6.5 lambda：只配做「一次性小函数」	36
6.6 函数是对象：回调与高阶函数	37
6.7 装饰器	38
第七章 类与对象	40
7.1 class 与 self	40
7.2 实例属性 vs 类属性	40
7.3 继承与 super()	41
7.4 property：把方法当属性读	41
7.5 classmethod 与 staticmethod	42
7.6 魔术方法：让自定义对象融入语言	44
7.7 dataclass：数据类的标准做法	45
7.8 组合优于继承：本项目的分层就靠它	46
第八章 异常处理	47
8.1 try 语句的四个部分	47
8.2 常见内置异常与层级	47
8.3 raise 与异常链 from	48
8.4 两种处理风格，以及怎么选	49
8.5 assert：只用来钉「不可能发生」	50
8.6 自定义异常	50
第九章 模块、包与导入	52
9.1 导入的四种写法	52

9.2 if <code>__name__ == "__main__"</code>	53
9.3 包、相对导入与 <code>__init__.py</code>	53
9.4 延迟导入：把 800 毫秒挪出启动路径	54
9.5 冻结（打包）后的路径	55
9.6 <code>sys.path</code> 与脚本运行	55
第十章 类型注解	56
10.1 注解写在三个地方	56
10.2 <code>Optional</code> 与联合类型	56
10.3 容器泛型	57
10.4 <code>Callable</code> ：把函数当参数传时怎么标注	57
10.5 <code>from __future__ import annotations</code>	58
10.6 注解不参与运行 —— 一条必须记住的边界	59
第十一章 标准库实战	60
11.1 <code>pathlib</code> ：路径就用 / 拼	60
11.2 <code>datetime</code> 与 <code>calendar</code> ：日期计算的三个必备套路	60
11.3 <code>json</code> ：结构化数据的通用交换格式	62
11.4 <code>sqlite3</code> ：本项目唯一的持久化方案	63
11.5 <code>logging</code> ：一份日志基建的最小正确实现	67
11.6 <code>re</code> ：正则的四个实用要点	68
11.7 其它常用模块	69
第十二章 GUI 相关的 Python 语法	71
12.1 界面为什么用类组织	71
12.2 回调：传函数对象，不传调用结果	71
12.3 <code>after</code> ：Tkinter 的定时器	72
12.4 事件绑定与事件对象	74
12.5 变量类： <code>StringVar</code> / <code>IntVar</code> / <code>BooleanVar</code>	74
12.6 控件生命周期与 <code>TclError</code>	75
12.7 一个跨语言的坑：Tk 会「饿死」控件	77
第十三章 工程约定与陷阱	78
13.1 行尾： <code>CRLF</code> 与 <code>LF</code>	78
13.2 文本 vs 字节：什么时候必须用字节	79
13.3 编码：统一 <code>UTF-8</code> ，一处都不要例外	79

13.4 时间：本项目明确选了「本机时间」	80
13.5 数值：先把误差摆到台面上	81
13.6 静态检查：pyflakes 与它的口径	81
13.7 打包：Python 特有的三个认知	82
附录 A • 语法速查卡	83
A.1 数据类型与字面量	83
A.2 字符串与容器操作	83
A.3 控制流与函数	83
A.4 类	84
A.5 模块与导入	84
A.6 常用标准库	85
A.7 高频惯用法	85
附录 B • 模块清单	87
B.1 数据目录结构	90
B.2 测试套件与基线	90
附录 C • 坑对照表	92
C.1 语法与语义类	92
C.2 文件与编码类	92
C.3 GUI 类	93
C.4 数据与打包类	93
附录 D • 工具脚本与开发流程	95
D.1 本项目自己的「补丁纪律」	95
D.2 这套流程的通用形状	96
D.3 待办（项目的下一步）	96
结语	96

第一章 手册说明与项目全景

这是一本**可以对着代码看的** Python 语法手册。它不讲「Python 是什么」，而是把语言特性一条条摆开，每条都配一段本项目（ServerTimeDemo / ExpiryManager）里正在运行的真实代码——你在项目里搜得到它、在手册里看得到它的语法依据，以及当初为什么写成那样。

这样编排的理由很实际：语法书上的例子总是干净的，而**真实的坑都长在「语法都对、组合起来就是不对」的地方**。本项目攒下的一批坑（行尾被改写、删除线画不出来、定时器断掉、目录页码不收敛）恰好都是这种，放在对应语法点旁边才记得住。

1.1 怎么用这本手册

- **当参考书查**：附录 A 是一页纸的语法速查表，附录 C 是「错误写法 → 正确写法」的对照表，两处都适合直接跳过去看。
- **当教材读**：第一到第十章按语言本身的顺序推进（词法 → 类型 → 控制流 → 函数 → 类 → 异常 → 模块 → 注解），每一节末尾都落到项目里的一段代码。
- **当项目导览读**：第十一章往后是「本项目真正用到的标准库」与「GUI 语法」、「工程约定与陷阱」，附录 B 是全模块清单。

版本对应关系

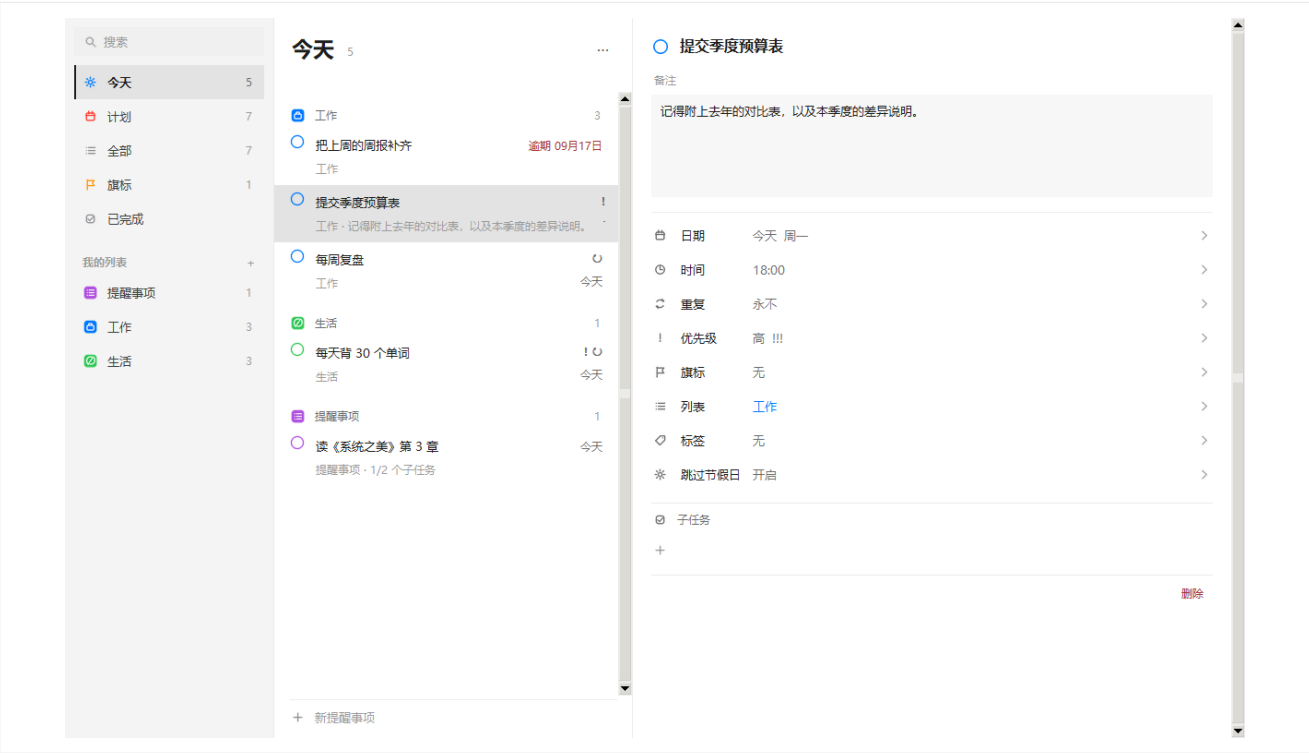
手册里引用的代码对应项目 v1.9.0。若代码后续改动，以仓库里的实际源码为准——附录 B 的模块清单与体量统计是**每次生成时现算的**，不会过时；正文里的代码是手抄的，改动后需要同步。

1.2 这个项目是什么

一句话：**一个单窗口的 Tkinter 桌面工作台**，把「服务器到期管理」「账号中心」「工具包」「学习笔记」「待办提醒」等一堆自用小功能收在一个程序里，用自绘的左侧导航切换，数据统一落在一个 SQLite 文件上。

它对学 Python 来说是个不错的样本，原因是三件事都齐了：

1. **语言面够宽**——数据类、装饰器、闭包、上下文管理、类型注解、推导式、生成器全都在用，不是玩具代码。
2. **有真实的持久化层**——24 张表的 SQLite，带迁移逻辑、事务、行工厂。
3. **有真实的界面层**——Tkinter 的坑几乎踩全了，注释里留着原因。



▲ 程序里的待办模块：左栏是清单（可拖动排序），中栏是条目列表，右栏是详情。手册里引用的界面代码大都能在这张图上找到对应形态。

1.3 代码规模

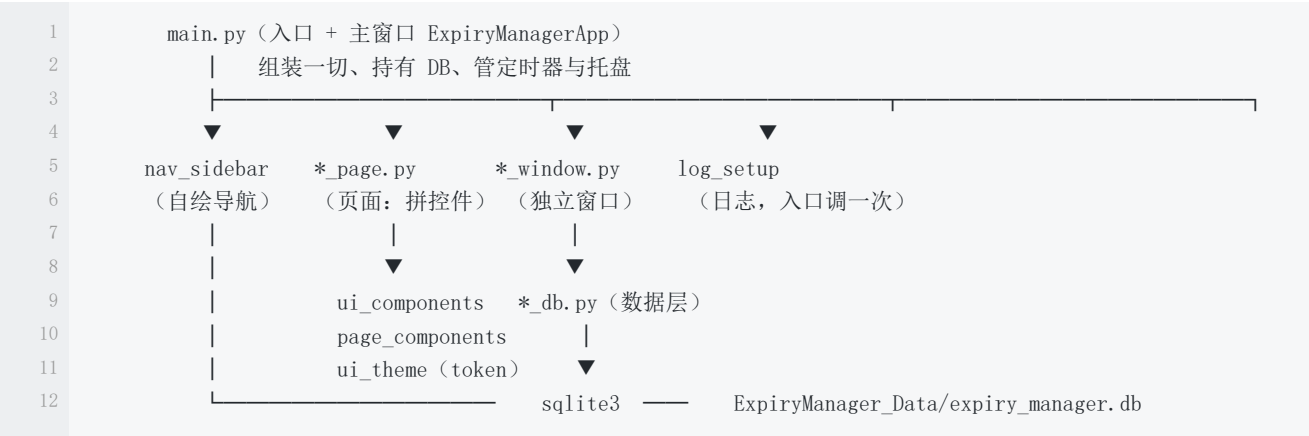
生成这份手册时统计的体量（每次生成现算，见 `scripts/make_python_manual.py`）：

位置	文件数	说明
根目录	34 个	业务模块：界面、数据层、主题、组件
<code>embedded_admin_tools/</code>	13 个	内置管理工具（按 <code>services/</code> 分层）
<code>scripts/</code>	27 个	开发期工具：测试、打包核对、手册生成本身

合计 74 个 Python 文件、44,314 行。根目录里 `main.py` 一个文件就有 5,400 余行 —— 它同时是「程序的入口」「主窗口的类定义」「一批通用工具函数」三个角色，这也是项目待办清单上「拆 `main.py`」那条的由来。

1.4 分层与数据流

模块之间的依赖是单向的，画成下来是这样：



▲ 分层约定：页面不直接写 SQL，数据层不认识 tkinter。

这条约定带来两个好处，也是项目里反复强调的：**数据层的模块可以在没有 tkinter 的解释器里跑测试**（本项目 11 个测试套件里有一批就是这么跑的），以及**换界面不用动数据逻辑**。

1.5 全模块地图

下面这张表覆盖根目录与 `embedded_admin_tools/` 的全部模块，职责取各文件自己的模块 docstring 首句 —— 想了解某个功能从哪个文件入手，查这里最快。

模块	行数	类	函数	职责
<code>main.py</code>	5444	9	39	ExpiryManager - 个人系统（到期管理器）v2.0
<code>tools_page.py</code>	4485	6	13	工具包页面（重写版：分类+包管理、相对路径、图标抽取）
<code>todo_page.py</code>	2165	3	3	待办事项模块 - 界面层（View Layer）
<code>study_notes_window.py</code>	1844	3	3	学习笔记主窗口
<code>embedded_admin_tools/api_demo_window.py</code>	1607	1	0	（无模块 docstring）
<code>scripts/test_tools_launcher.py</code>	1576	0	36	系统工具箱「启动器」回归测试。
<code>todo_db.py</code>	1545	5	10	待办事项模块 - 数据层（Data Layer）
<code>embedded_admin_tools/market_quote_window.py</code>	1458	1	0	（无模块 docstring）
<code>scripts/test_todo_ui.py</code>	1355	1	29	待办页面 UI 回归测试：真实窗口 + 样例数据 + 逐项断言。
<code>adb_page.py</code>	1080	1	0	ADB 工具箱
<code>scripts/test_todo.py</code>	976	0	17	待办 / 提醒事项模块回归测试（数据层 + 节假日引擎，纯逻辑、不碰 Tk）。
<code>embedded_admin_tools/error_code_service.py</code>	929	1	2	错误码服务 - <code>returnCode.json</code> 内嵌版本

模块	行数	类	函数	职责
<code>study_demo_window.py</code>	925	2	0	Python 学习辅助模块
<code>study_notes_db.py</code>	895	4	2	学习笔记模块 - 数据层 (Data Layer)
<code>scripts/manual_engine.py</code>	838	3	14	Python 语法手册的排版引擎 —— 把「块列表」渲染成一份带目录、页码、书签的 PDF。
<code>qa_work_log_page.py</code>	774	5	0	Q&A 问答 + 每周工作纪要 + 练习记录 页面
<code>tools_db.py</code>	768	1	38	工具库数据库
<code>qa_work_log_db.py</code>	707	8	4	Q&A 问答 + 每周工作纪要 + 练习记录 数据层
<code>qa_extras.py</code>	668	4	4	Q&A + 工作纪要 + 练习记录 扩展功能
<code>ui_theme.py</code>	592	3	0	全局 UI 主题定义。
<code>embedded_admin_tools/crypto_window.py</code>	573	1	0	(无模块 docstring)
<code>account_windows.py</code>	542	2	1	共享账户管理相关窗口。
<code>embedded_admin_tools/interface_service.py</code>	527	1	2	接口服务 - interfaces.json 内嵌版本
<code>markdown_view.py</code>	509	0	5	Markdown 渲染 (Typora 风格)
<code>expiry_dialogs.py</code>	492	3	2	到期管理相关弹窗。
<code>ui_components.py</code>	482	2	10	统一 UI 组件层。
<code>embedded_admin_tools/login_checker_window.py</code>	464	1	0	(无模块 docstring)
<code>baidu_disk_window.py</code>	463	2	0	百度网盘资料库管理窗口
<code>tools_launcher.py</code>	408	0	23	<code>tools_launcher.py</code> —— 工具启动器的纯逻辑层。
<code>embedded_admin_tools/app_icon_service.py</code>	398	0	1	主程序图标内嵌服务
<code>console_page.py</code>	397	1	0	<code>console_page.py</code> - 控制台页面(日志输出 + 命令执行 + 工具维护)
<code>todo_icons.py</code>	375	0	47	待办模块的自绘图形资产。
<code>app_icons.py</code>	373	1	23	应用图形资产: 应用标记 + 导航线性图标。
<code>scripts/project_audit.py</code>	367	1	10	项目静态审计脚本 — 只读分析, 不修改任何被分析文件。
<code>scripts/prune_unused_imports.py</code>	343	0	10	未使用导入裁剪器 —— 依据 pyflakes 结果精确删除导入名, 只读预览为默认。
<code>nav_sidebar.py</code>	336	1	0	左侧分组导航侧栏 (自绘)。

模块	行数	类	函数	职责
<code>scripts/test_notes_markdown.py</code>	323	0	6	笔记页排版回归测试：Markdown 预览 tag + 编辑器外框 + 底部预览面板。
<code>scripts/test_notes_editor.py</code>	304	1	3	笔记编辑器回归测试：格式工具栏动作 + 源码淡化 + 语法速查 + 两处预览渲染。
<code>scripts/make_python_manual.py</code>	295	0	9	生成《Python 语法手册（配 ServerTimeDemo 项目实例）》PDF。
<code>scripts/test_detail_dialog.py</code>	288	0	7	资源详情弹窗回归测试。
<code>app_version.py</code>	285	0	3	版本号与更新日志的单一数据源。
<code>scripts/make_app_icon.py</code>	285	0	10	生成应用图标（app.ico），并同步更新内嵌 base64 副本。
<code>scripts/test_expiry_table.py</code>	277	0	4	到期表格「单元格收敛」回归测试。
<code>study_demo_db.py</code>	271	0	20	Python 学习模块数据库层
<code>backend_page.py</code>	244	1	0	后台首页页面类。
<code>scripts/check_name_scope.py</code>	244	0	7	名字与作用域检查 —— 只读分析，不修改任何被检查文件。
<code>scripts/test_shell_ui.py</code>	242	0	3	外壳回归测试：自绘侧栏 + 顶栏 + 主题 token 的运行时验证。
<code>credential_process_dialogs.py</code>	233	3	0	账号中心与流程中心相关弹窗。
<code>scripts/scan_source_writers.py</code>	215	0	3	源码写入扫描 —— 找出会「就地重写源码」的脚本（只读分析，绝不修改文件）。
<code>scripts/runtime_smoke.py</code>	214	0	2	运行时冒烟测试 —— 真实执行关键代码路径，不依赖静态推断。
<code>embedded_admin_tools/login_memory_service.py</code>	212	2	1	（无模块 docstring）
<code>scripts/test_release_meta.py</code>	210	0	7	发布元信息回归：版本号 / 更新日志 / CHANGELOG.md 是否一致。
<code>scripts/test_log_setup.py</code>	203	0	2	log_setup 回归测试 —— 真实写文件、真实读回，逐条断言。
<code>credentials_page.py</code>	195	1	0	账号中心页面类。
<code>expiry_page.py</code>	185	1	5	到期管理页面类。
<code>log_setup.py</code>	176	0	5	全局日志配置 —— 在程序入口调用一次，全项目模块自动生效。
<code>page_components.py</code>	176	0	10	页面级组件层。
<code>dialog_form_style.py</code>	167	0	7	弹窗表单统一样式。

模块	行数	类	函数	职责
<code>process_page.py</code>	152	1	0	流程中心页面类。
<code>scripts/check_exe.py</code>	147	0	2	打包产物体检：exe 时间戳 / 体积 + 从归档里反向核对源码真的进去了。
<code>scripts/window_shot.py</code>	139	1	3	把 Windows 窗口抓成 PNG —— 用 <code>PrintWindow</code> ，不依赖桌面是否可见。
<code>scripts/dep_inventory.py</code>	137	0	2	依赖清单盘点 —— 静态提取全项目 <code>import</code> ，分类为 标准库 / 第三方 / 项目内。
<code>scripts/run_all_tests.py</code>	126	0	2	一条命令跑完全部回归套件，并汇总成一张表。
<code>scripts/verify_icon_generators.py</code>	113	0	3	图标生成器等价性验证 —— 只读比对，不写入任何目标文件。
<code>scripts/gen_app_icon_service.py</code>	69	0	2	由 <code>app.ico</code> 生成内嵌图标的服务模块。
<code>scripts/gen_app_icon.py</code>	61	0	2	生成主程序图标 <code>app.ico</code> （简化版占位图）。
<code>scripts/gen_icon_service.py</code>	61	0	1	由 <code>base64</code> 文本生成内嵌图标的服务模块。
<code>embedded_admin_tools/api_client.py</code>	55	1	0	（无模块 <code>docstring</code> ）
<code>scripts/gen_changelog.py</code>	50	0	2	从 <code>app_version.VERSION_HISTORY</code> 生成仓库根目录的 <code>CHANGELOG.md</code> 。
<code>embedded_admin_tools/crypto_service.py</code>	22	1	0	（无模块 <code>docstring</code> ）
<code>embedded_admin_tools/error_code_service.py</code>	7	0	0	兼容包装：统一转发到 <code>embedded_admin_tools.services.error_code_service</code> 。
<code>embedded_admin_tools/interface_service.py</code>	7	0	0	兼容包装：统一转发到 <code>embedded_admin_tools.services.interface_service</code> 。

1.6 阅读约定

记号	含义
等宽字	代码、文件名、函数名、命令行
粗体	关键结论
▲ 出处	紧跟代码块下方，表明这段代码来自哪个文件的哪个函数
灰底方框	引自项目源码 <code>docstring</code> 的原文，是我自己当初写的注释
蓝灰色提示框	背景知识或设计取舍
橙边框警告框	踩过的坑，以及为什么不能那么写
绿边框结论框	拍板下来的正确做法

代码块左侧是对齐的行号，方便你在源码里定位；行号栏与代码栏底色不同，复制代码时不容易把行号一起带走。

第二章 词法与基础语法

这一章讲最小的单位：一个源文件长什么样、注释怎么写、名字怎么起、行怎么折。这些都是「一眼就能看懂、但写错了要很久才发现」的东西。

2.1 文件头：编码声明与模块 docstring

Python 3 的源文件默认就是 UTF-8，所以 `# -*- coding: utf-8 -*-` 严格来说不是必需的。项目里每个文件仍然带着它——这是延续下来的习惯，好处是文件被拿去看、被别的编辑器打开时意图明确。

真正重要的是紧跟其后的模块 **docstring**：它是文件的第一个语句，会被 `help()` 和文档工具读到。本项目把它当「文件级说明书」用，把这张表有哪些表、有哪些容易误解的语义都写在里面：

```
1 # -*- coding: utf-8 -*-
2 """
3 =====
4 待办事项模块 - 数据层 (Data Layer)
5 =====
6 本模块封装「待办 / 提醒事项」所需的全部持久化逻辑与**工作日日历**推算。
7 复用主程序的 SQLite 数据库文件 (db_path 由调用方注入)，与其余模块共享连接，
8 因此备份、迁移都只需处理一个文件。
9
10 本模块涉及的表（建表语句见 create_tables）：
11     todo_lists      清单：名称、颜色（苹果 12 色之一）、图标、排序
12     todo_items      待办条目：标题、备注、日期、时间、重复、优先级、旗标、标签
13     todo_subtasks   子任务：挂在条目下的轻量清单项
14     todo_holidays   节假日日历：放假 / 调休上班两天都记在这一张表
15     todo_state      模块内键值状态：上次工作日弹窗日期等
16
17 工作日语义（这层是「周末 / 节假日自动跳过」的全部依据）
18 =====
19 本模块把「工作日」定义为：**同时不是法定放假日、也不是周六周日**的那一天；
20 唯一的例外是**调休上班日**——它虽然落在周末，但因为被国务院安排上班，
21 所以算工作日。
22 """
```

▲ todo_db.py 开头 · 节选（原文用表格画表结构，此处为放进版心改排成列表）

docstring 的三种写法

单行：“”一句话说明。“”收在同行；**多行**：首行一句话概要、空一行、再写细节，结尾的“”单独一行（PEP 257）；**原始串**：内容里有反斜杠时用 `r"""..."""`，避免转义被解释。本项目的模块 docstring 统一是多行写法，因为要塞进表结构和语义约定。

2.2 注释：为什么这么写

本项目的注释有个明确取向：**不解释「这行在做什么」，只解释「为什么不是别的写法」**。代码本身已经说明前一半，后一半才是三个月后还会绊人的部分。

```
1 def _migrate_alert_columns(self):
2     """给老库补上「到点提醒」的两列。
3
4     新装的程序建表时就带上它们，但**已经装过老版本**的库里
5     ``todo_items`` 没有这两列 —— SQLite 又没有
6     ``ADD COLUMN IF NOT EXISTS``，只能先查 ``PRAGMA table_info``
7     再按需补，否则用户得删库重来。
8     """
9     existing = {
10         row["name"]
11         for row in self.conn.execute("PRAGMA table_info(todo_items)").fetchall()
12     }
13     for column, ddl in (
14         ("alerted_for", "ALTER TABLE todo_items ADD COLUMN "
15                        "alerted_for TEXT NOT NULL DEFAULT ''"),
16         ("snooze_until", "ALTER TABLE todo_items ADD COLUMN "
17                        "snooze_until TEXT NOT NULL DEFAULT ''"),
18     ):
19         if column not in existing:
20             self.conn.execute(ddl)
21     self.conn.commit()
```

▲ todo_db.py · TodoDB._migrate_alert_columns()

注意这里没有一行注释在说「查询表结构」「循环补列」。它说的是**为什么不能写** `ADD COLUMN IF NOT EXISTS`、以及**漏了会有什么后果**（用户得删库重来）。

2.3 命名规范

Python 社区只有一份约定（PEP 8），没有强制。本项目的实际用法如下，照着写就不会有突兀感：

对象	写法	项目里的例子
模块	全小写 + 下划线	todo_db、nav_sidebar、page_components
包	全小写，不用下划线	embedded_admin_tools
类	大驼峰 CapWords	TodoDB、TodoPage、ScrollArea、ManualDoc
函数 / 方法	全小写 + 下划线	pending_alerts、_migrate_alert_columns
变量	全小写 + 下划线	due_date、submit_text、moment
常量	全大写 + 下划线	ALERT_GRACE_MINUTES、TODO_TICK_MS
私有成员	前导单下划线	self._conn、_norm、_STRIKE_FONTS
类型变量	同变量	list[str]、dict[int, str]
「别碰」的成员	双下划线前后	__init__、__eq__、__enter__

单下划线与双下划线的区别

前导单下划线 (`_norm`) 只是「约定别在外部用」，`from m import *` 会跳过它，但直接写 `m._norm()` 照样能调——它是给人的信号，不是给解释器的锁。**前导双下划线** (`self.__x`) 会触发名称改写 (mangling)，在类 `Foo` 里变成 `_Foo__x`，用于避免与子类同名属性打架。本项目只用单下划线，因为**界面层的测试需要直接点私有方法**（例如探针里调 `page._list_drag_press(...)` 模拟拖动），双下划线反而会让测试写不下去。

2.4 缩进与续行

缩进用 **4 个空格**，项目里没有一处用 Tab。行太长时有三种合法折法，本项目三种都在用：

- 1. **括号里直接换行**（隐式续行，最常用）：`( [ {` 之内不需要任何续行符，这是首选写法，因为它不依赖行尾没有空格。
- 2. **行尾反斜杠**：只有在括号外、又必须折行时才用。项目里几乎不用——反斜杠后粘一个空格就是语法错误，而肉眼看不出来。
- 3. **把长表达式提成变量或辅助函数**：比前两种都好，本项目后来倾向这种。

```
1     for column, ddl in (  
2         ("alerted_for", "ALTER TABLE todo_items ADD COLUMN "  
3             "alerted_for TEXT NOT NULL DEFAULT ''"),  
4         ("snooze_until", "ALTER TABLE todo_items ADD COLUMN "  
5             "snooze_until TEXT NOT NULL DEFAULT ''"),  
6     ):  
7         if column not in existing:  
8             self.conn.execute(ddl)
```

▲ 括号内的隐式续行 · `todo_db.py`

上面这段还有一处值得留意的细节：两个字符串字面量**直接相邻** (`"ALTER TABLE ... "` 和 `"alerted_for ... "`)，Python 会在编译期把它们拼成一个，不会产生运行期开销——这是拆长字符串的惯用手段。

2.5 运算符速查

类别	运算符	说明 / 项目里的用法
算术	<code>+ - * /</code>	<code>/</code> 永远得到 <code>float</code> ，即使能整除
整除 / 取余	<code>// %</code>	<code>divmod(总数, 12)</code> 一次拿到商和余 (<code>add_months()</code>)
幂	<code>**</code>	<code>2 ** 10</code>
比较	<code>== != < <= > >=</code>	可以链式写： <code>0 <= hour <= 23</code>
身份	<code>is / is not</code>	只与 <code>None / True / False</code> 比较时用
成员	<code>in / not in</code>	<code>if column not in existing</code>
逻辑	<code>and or not</code>	短路求值； <code>not</code> 优先级最低，别省括号
位运算	<code>& ^ ~ << >></code>	本项目用在权限位与颜色通道上

类别	运算符	说明 / 项目里的用法
赋值	<code>= += -= *= /= //= %= **=</code>	还有 <code>&= =</code> 等位运算版本
三目	<code>A if 条件 else B</code>	表达式，不是语句
海象	<code>:=</code>	在表达式里赋值，见 5.6
解包	<code>*</code> / <code>**</code>	<code>f(*args, **kwargs)</code> 、 <code>a, *rest = seq</code>

`0 <= hour <= 23` 为什么在 Python 里能写

这是链式比较，Python 把它编译成 `0 <= hour and hour <= 23`，且中间的 `hour` 只求值一次。项目里解析时间就是这么校验的：`if not (0 <= hour <= 23 and 0 <= minute <= 59): return None`。

2.6 为什么这里必须用字节而不是字符

本项目有一条贯穿始终的纪律：**改源码文件必须走字节层**。先看现状——项目里不同文件的行尾是不一样的：

文件	行尾现状	成因
应用源码 (<code>main.py</code> 、 <code>todo_db.py</code> ...)	CRLF	在 Windows 上生成、被编辑器改过
<code>main.py</code>	混排：CRLF + 少量单独 LF	历史上被不同工具改过
<code>scripts/*.py</code>	LF	在 Git Bash 环境里生成
<code>*.md</code>	LF	同上

行尾混排本身不影响运行，但会把 **diff 炸掉**：如果把 `main.py` 用普通文本方式写回去，Python 的默认行为会把整个文件的行尾统一成一种，于是「改 5 行」在 `git diff` 里显示为「改了 5,000 行」。所以本项目改源码的姿势是先读进来、只替换目标片段、再原样写回：

```
1 from pathlib import Path
2
3 p = Path("todo_db.py")
4 raw = p.read_bytes()           # 字节读，不经过文本层
5 assert raw.count(b"\r\n") == raw.count(b"\n")  # 确认是纯 CRLF
6 before = raw.count(b"\n")
7 text = raw.decode("utf-8").replace("\r\n", "\n")  # 统一成 LF 再操作
8
9 # .....在这里对 text 做替换，并断言每处锚点恰好命中一次.....
10
11 out = text.replace("\n", "\r\n").encode("utf-8")
12 assert out.count(b"\n") == before + added      # 行数变化符合预期
13 assert out.count(b"\r\n") == out.count(b"\n")  # 没有意外的单独 LF
14 p.write_bytes(out)                             # 字节写，不经文本层
```

▲ 项目里改源码通用的「字节级补丁」骨架

这条纪律的三条硬要求

① 每处锚点**断言恰好命中一次**，否则整体拒写（避免半截替换）；② 断言**行尾变化量**与预期一致；③ 写完用 `py_compile` 与 `git diff --numstat` 复核变更行数。三条齐了，才敢按下执行。

第三章 变量与内置类型

Python 的变量不是「装东西的盒子」，而是**贴在对象上的名字**。理解这一点，后面很多现象（可变默认参数、闭包捕获、`is` 与 `==` 的区别）就都有了解释。

3.1 赋值与解包的各种形态

写法	效果
<code>a = b = 0</code>	两个名字指向同一个对象
<code>a, b = 1, 2</code>	元组解包，右边先构造元组
<code>a, b = b, a</code>	交换（右边先整体求值）
<code>first, *rest = seq</code>	带星号解包， <code>rest</code> 是列表
<code>a, (b, c) = 1, (2, 3)</code>	嵌套解包
<code>x += 1</code>	对不可变对象等于 <code>x = x + 1</code> ；对列表是原地修改

项目里最常见的解包是**遍历字典 / 元组列表时当场拆开**：

```
1 REPEAT_RULES: list[tuple[str, str]] = [  
2     ("none", "永不"),  
3     ("daily", "每天"),  
4     ("weekly", "每周"),  
5     ("biweekly", "每两周"),  
6     ("monthly", "每月"),  
7     ("yearly", "每年"),  
8     ("custom", "自定"),  
9 ]  
10 REPEAT_LABELS = dict(REPEAT_RULES)           # 元组列表 -> 字典，一行搞定
```

▲ `todo_db.py` · 规则表与其反查字典同源

`dict()` 接一个「二元组序列」就能构造字典，所以下拉框要的顺序（列表）和按值查标签（字典）可以共用一份数据，永远不会不同步。

3.2 数字：int / float / bool

`int` 是任意精度的（不会溢出），`float` 是 IEEE-754 双精度（**会有误差**），`bool` 是 `int` 的子类（`True == 1`）。

```
1 def excel_serial_to_date(value: float) -> date:  
2     """Excel 的日期序列号转 date：以 1899-12-30 为原点。"""  
3     base = datetime(1899, 12, 30)  
4     return (base + timedelta(days=float(value))).date()
```

▲ `main.py` · 导入 Excel 时的日期换算

金额和精度的规矩

0.1 + 0.2 == 0.3 是 **False** (结果是 0.30000000000000004) 。所以：① 金额一律用 `int` 存「分」，或 `decimal.Decimal`；② 比较浮点用 `abs(a - b) < 1e-9`，别用 `==`；③ 展示时用 `f"{v:.2f}"` 格式化，不要指望它精确。本项目目前的金额场景都还在 Excel 导入导出环节，因此是「转成 `float` 再格式化」，没有做分单位存储——涉及记账时要改。

3.3 字符串

字符串是**不可变**的：任何「修改」都是造了一个新串。这决定了「循环里拼字符串」是 $O(n^2)$ 的坏习惯，应该先收进列表再 `join`。

三种字符串字面量

```
1 plain = "ALTER TABLE todo_items ADD COLUMN alerted_for TEXT NOT NULL DEFAULT ''"
2 fstring = f"有 {len(missed)} 条提醒已过时间: {names} {tail}"
3 raw_text = r"C:\Users\shaoy\AppData"          # r 前缀：反斜杠不转义
4 multiline = """第一行
5 第二行"""
```

▲ 三种字面量；注意 f-string 里的单引号是 SQL 里的空串，不是 Python 的

f-string 的常用写法

写法	结果
<code>f"{name}"</code>	直接插入
<code>f"{value:.2f}"</code>	保留两位小数
<code>f"{n:>3}"</code>	右对齐、宽度 3（拼菜单与行号时常用）
<code>f"{raw!r}"</code>	用 <code>repr()</code> 的结果插入，调试时看得到引号与转义
<code>f"{when:%Y-%m-%d}"</code>	直接套日期格式
<code>f"{items=}"</code>	3.8+ 的调试写法，展开成 <code>items=[1, 2]</code>

```
1 names = ", ".join(it.title for it in missed[:3])
2 tail = f" 等 {len(missed)} 条" if len(missed) > 3 else ""
3 self.tray.notify(APP_TITLE, f"有 {len(missed)} 条提醒已过时间: {names} {tail}")
```

▲ `main.py` · 托盘汇总文案；`join` 接的是生成器表达式，不需要先造列表

常用方法与切片

写法	作用
<code>s.strip()</code>	去掉两端空白（还有 <code>lstrip</code> / <code>rstrip</code> ）
<code>s.split(" ")</code>	按分隔符切成列表；不传参数则按任意空白切并去掉空串

写法	作用
<code>s.replace(a, b)</code>	全部替换
<code>s.startswith(p) / s.endswith(p)</code>	前后缀判断
<code>s.casefold()</code>	大小写无比较用的规范化（比 <code>lower()</code> 更彻底）
<code>s.zfill(2)</code>	补零到两位，如 <code>"9".zfill(2) == "09"</code>
<code>s[:2]</code>	取前两个字符（切片不会越界报错）
<code>"-".join(seq)</code>	拼接，元素必须都是字符串
<code>s in other</code>	子串判断

```

1 def _norm(value) -> str:
2     """把任意值规范化为去空白字符串；None 视作空串。"""
3     if value is None:
4         return ""
5     return str(value).strip()

```

▲ `todo_db.py · _norm()`，全模块 30 多处在用

`_norm` 为什么值得单独存在

SQLite 取出来的列可能是 `None`、可能是 `""`、也可能是带空格的文本。如果每个字段各自 `strip()`，迟早有人漏掉一处，于是出现「明明填了日期却说没填」。收成一个函数之后，「空」在整个模块里只有一个含义：空字符串。这个函数还顺带承担了类型宽容——传进来数字也能变成字符串。

3.4 真值判定与 `None`

任何对象都有「真假」。下面这些**都是假**：`None`、`False`、`0`、`0.0`、`""`、`[]`、`()`、`{}`、`set()`。其余为真。

```

1 if not _norm(item.due_date) or not _norm(item.due_time):
2     return None           # 没设日期或时间 -> 这条根本不参与提醒

```

▲ `todo_db.py · alert_moment()` 的第一道闸

因为空串是假值，上面这两句既可以理解为「没设置」，也可以理解为「设置了但是空的」——两者在业务上是同一件事，正好不需要区分。

or 兜底的惯用法

```

1 color=_norm(row["color"]) or DEFAULT_LIST_COLOR,
2 repeat_rule=_norm(row["repeat_rule"]) or "none",
3 sort_order=int(row["sort_order"] or 0),

```

▲ `todo_db.py · from_row()` 里成排的兜底

`a or b` 的含义是「`a` 若为假就取 `b`」，正好用来填默认值：库里的空值 → 变成有意义的默认值。**但要注意它同时会吞掉合法的 0 与空串**——所以这里的 `int(row["sort_order"] or 0)` 是安全的（0 本来就是想要的默认值），而如果某个字段的合法值包含 0，就不能这么写。

与 None 比较：用 is

写法	结论
<code>if x is None:</code>	推荐。 <code>None</code> 是单例， <code>is</code> 是身份比较，没有重载歧义
<code>if x == None:</code>	能用但不推荐； <code>x</code> 的自定义 <code>__eq__</code> 可能改变结果
<code>if not x:</code>	判「假值」。当心它会连 0、空串、空列表一起匹配
<code>if x:</code>	判「有值」

```
1 dialog = getattr(self, "_todo_alert_win", None)
2 self._todo_alert_win = None
3 if dialog is None:
4     return
```

▲ `main.py · _close_todo_alert()`：先取出、先置空、再判断

这里刻意分了三步。如果写成 `if self._todo_alert_win: self._todo_alert_win.destroy()` 再置空，第二层调用（把引用清空后）也不会出错，但**中间任何一步抛异常都会留下一个指向已销毁控件的引用**，而 `destroy()` 之后的控件再访问会抛 `TclError: invalid command name`。先把引用置 `None`，等于让「关闭」这个操作**幂等**——重复调用安全。

3.5 类型转换

写法	结果 / 注意
<code>int("42")</code>	42; <code>int("4.2")</code> 会抛 ValueError
<code>int(4.9)</code>	4, 向零取整 ，不是四舍五入
<code>float("4.2")</code>	4.2
<code>str(42)</code>	'42'; 对对象是 <code>repr</code> 还是 <code>str</code> 取决于是否定义了 <code>__str__</code>
<code>bool([])</code>	False
<code>list("abc")</code>	['a', 'b', 'c']
<code>list({"a": 1})</code>	['a'] —— 字典转列表拿到的是键
<code>dict([("a", 1)])</code>	{'a': 1}
<code>round(2.675, 2)</code>	2.67 ，浮点误差所致，别指望它四舍五入

`int()` 的三种失败方式

`int("")`、`int("abc")`、`int(None)` 全都抛 **ValueError** 或 **TypeError**。所以从界面或文件读来的数字一律要防：本项目用的姿势是 `int(x)` 外面套 `try / except ValueError`，或者用 `or` 先兜成默认值 —— `int(row["sort_order"] or 0)` 里的 `or 0` 就是为了让 `None` 不进 `int()`。

第四章 容器类型

四种内建容器划分得很清楚：**list 有序可变**、**tuple 有序不可变**、**dict 按键取值**、**set 只关心「有没有」**。选错的典型症状是「明明只想判断存在性，却写了一个每次都要遍历的列表」。

类型	字面量	可变	有序	适用
list	<code>[1, 2]</code>	是	是	有顺序的一批东西；界面的行、排序结果
tuple	<code>(1, 2)</code>	否	是	固定的组合；能当 dict 的键
dict	<code>{"a": 1}</code>	是	是 (3.7+ 保证插入序)	按名字取东西；配置、行数据
set	<code>{1, 2}</code>	是	否	去重、存在性判断、集合运算

注意 `{}` 是空字典，空集合要写 `set()`。

4.1 list：本项目的界面行就是它

操作	写法
追加 / 扩展	<code>items.append(x)</code> / <code>items.extend(seq)</code>
插入 / 删除	<code>items.insert(i, x)</code> / <code>del items[i]</code> / <code>items.pop()</code>
查找	<code>items.index(x)</code> (找不到抛 <code>ValueError</code>)、 <code>x in items</code>
排序	<code>items.sort()</code> (原地) vs <code>sorted(items)</code> (新列表)
倒序 / 反转	<code>items.reverse()</code> (原地)、 <code>list(reversed(items))</code>
浅复制	<code>items.copy()</code> 或 <code>items[:]</code>
统计	<code>items.count(x)</code> 、 <code>len(items)</code> 、 <code>sum(nums)</code> 、 <code>max / min</code>

```
1 due: list[TodoItem] = []
2 missed: list[TodoItem] = []
3 moments: dict[int, str] = {}
4 for row in rows:
5     item = TodoItem.from_row(row)
6     moment = alert_moment(item)
7     if moment is None or moment > now:
8         continue
9     text = moment_str(moment)
10    (missed if now - moment > grace else due).append(item)
11    moments[item.id] = text
```

▲ `todo_db.py · pending_alerts()` 内部（此处略作合并以缩短篇幅）

`(a if 条件 else b).append(x)` 这种写法能省掉一个 `if` 分支，但**只在两个候选都很短时才值得**。更长的情况宁可写两行——本项目在 `_run_todo_alerts` 里就是老老实实分成 `if missed:` 与 `if due:` 两段。

4.2 tuple: 固定结构与「可哈希」

元组常被当成「不可变的列表」，其实它更适合**表示一条结构固定的记录**——比如「颜色名 + 色值」、「标签 + 个数」。它还有一个列表没有的能力：**可以当字典的键、可以放进 `set`**，因为它不可变、可哈希。

```
1 # 苹果提醒事项的 12 色列表色（取自 iOS 18 自建列表取色盘）
2 LIST_COLORS: list[tuple[str, str]] = [
3     ("红色", "#FF3B30"),
4     ("橙色", "#FF9500"),
5     ("黄色", "#FFCC00"),
6     ("绿色", "#34C759"),
7     ("薄荷", "#00C7BE"),
8     ("蓝色", "#007AFF"),
9     ("靛蓝", "#5856D6"),
10    ("紫色", "#AF52DE"),
11    ("玫红", "#FF2D55"),
12    ("棕色", "#A2845E"),
13    ("深灰", "#8E8E93"),
14    ("浅褐", "#D4B89F"),
15 ]
```

▲ `todo_db.py` · `LIST_COLORS`

这里为什么不用两个平铺的列表

写成 `NAMES = [...]` 加 `VALUES = [...]` 也能跑，但那样「第 3 个颜色叫什么」要靠下标对齐，插一个就会错位。**两个互相有对应关系的东西，就装进同一个元组**，让它们物理上无法分离。

4.3 dict: 本项目的配置与行数据主力

操作	写法	注意
取值（带默认）	<code>d.get(k, default)</code>	键不存在也不报错 ； <code>d[k]</code> 会抛 <code>KeyError</code>
存在性	<code>k in d</code>	判断的是键，不是值
设默认值	<code>d.setdefault(k, v)</code>	键在就不动，不在才写入
更新多个	<code>d.update(other)</code>	或 3.9+ 的 <code>d1 d2</code> （合并出新字典）
删除	<code>del d[k]</code> / <code>d.pop(k, None)</code>	<code>pop</code> 给默认值可以不报错
遍历	<code>d.items()</code> / <code>d.keys()</code> / <code>d.values()</code>	遍历时 不能改大小
构造	<code>dict(zip(keys, vals))</code>	两个列表配成字典

```

1 def popular_tags(tools, limit: int = 0) -> list[tuple[str, int]]:
2     """统计一批工具的标签使用次数: [(标签, 个数)], 个数降序、标签升序。"""
3     counts: dict[str, int] = {}
4     label: dict[str, str] = {}
5     for tool in tools:
6         for tag in tool_tags(tool):
7             key = tag.casefold()
8             counts[key] = counts.get(key, 0) + 1
9             label.setdefault(key, tag)
10    items = [(label[key], count) for key, count in counts.items()]
11    items.sort(key=lambda item: (-item[1], item[0].casefold()))
12    return items[:limit] if limit > 0 else items

```

▲ tools_launcher.py · popular_tags()

这一小段里有四个可以直接拿去用的字典惯用法：

1. **计数**: `counts[k] = counts.get(k, 0) + 1`。不用先判断键在不在。
2. **保留首次出现的写法**: `label.setdefault(k, tag)` —— 大小写不同的标签 (`Python / python`) 只算一个, 展示时用第一次遇到的写法。
3. **两张表同源**: 计数用规范化后的键, 展示用原始写法, 各司其职。
4. `items()` **遍历**: 拿到的是 (键, 值) 二元组, 直接就地解包。

不要在遍历字典时增删键

`for k in d: del d[k]` 会抛 `RuntimeError: dictionary changed size during iteration`。正确做法是先取快照：`for k in list(d): ...`, 或者收集成列表再统一删。本项目在 `log_setup._reset_for_tests` 里就是先 `list(root.handlers)` 再逐个移除。

4.4 set: 判断「有没有」比 list 快得多

`x in list` 要逐个比对 ($O(n)$) , `x in set` 走哈希 (平均 $O(1)$) 。只要一段代码反复问「这个在里面吗」, 就该换成 **set** —— 本项目在数据库迁移里就是这么用的:

```

1     existing = {
2         row["name"]
3         for row in self.conn.execute("PRAGMA table_info(todo_items)").fetchall()
4     }
5     for column, ddl in (...):
6         if column not in existing:
7             self.conn.execute(ddl)

```

▲ todo_db.py · 先把列名收成 set, 再逐列判断

这里其实只判断了 2 次, 换不换 set 无所谓; 但同一个模式在 `main.py` 里判断 `tool_items / shared_accounts / credential_items / users` 四张表几十个列时, 就是在真省时间。统一的写法让「哪种都行」的地方也不用挑。

集合运算

写法	含义
<code>a b</code>	并集
<code>a & b</code>	交集
<code>a - b</code>	差集（在 a 不在 b）
<code>a ^ b</code>	对称差（只在一侧）
<code>a <= b</code>	a 是否为 b 的子集
<code>a.isdisjoint(b)</code>	两者是否完全不相交

4.5 推导式：一行构造容器

推导式（comprehension）是 Python 最有辨识度的语法之一。四种形态：

```
1 names = [row["name"] for row in rows]           # 列表推导
2 names = [n for n in names if n]                 # 带过滤
3 sizes = {name: len(name) for name in names}     # 字典推导
4 unique = {name.casefold() for name in names}    # 集合推导
5 total = sum(len(name) for name in names)        # 生成器表达式（不是推导）
```

▲ 四种形态

生成器表达式与列表推导只差一对方括号，但它不一次性造出列表，而是按需产出。传给 `sum / join / any / all` 这类函数时应该用生成器版本，省一次内存分配：

```
1 names = "\n".join(it.title for it in missed[:3])
```

▲ `main.py` · `join` 直接吃生成器，不需要先造列表

项目里也大量用推导式做「提取一列」的活：

```
1 headers = [normalize_text(value) for value in header_row]
2 header_map = {header: index for index, header in enumerate(headers) if header}
3 cols = [p.name for p in sorted(rows, reverse=True)[:20]]
4 labels = [item["label"] for item in parse_account_image_items(image_value)
5           if item.get("label")]
```

▲ 分别出自 `main.py` 与手册生成本身

推导式别写太长

推导式一旦超过一行、或者带两三个 `if`，可读性会迅速差过普通 `for` 循环。本项目的规矩是：**推导式里不写函数调用以外的逻辑**；需要分支就用普通循环 + `append`。判断标准很朴素 —— 「这行字能不能一口气念完」。

4.6 排序与 key

`sort()` 与 `sorted()` 都接受 `key=` 参数：**key 是一个函数，负责把元素映射成「拿来比较的东西」**。按多个条件排序，就让 key 返回元组。

```
1 scored.sort(key=lambda item: (-item[0], item[1]))
```

▲ `tools_launcher.py` · 先按分数降序、分数相同再按原顺序升序

`-item[0]` 是「降序」的常用写法（元组比较是逐项比较，负号翻转了大小关系）。第二项 `item[1]` 是原始下标，用它保证**同分时顺序稳定**，否则每次搜索结果都会自己换位置。

key 也可以是一个具名函数，条件复杂时比 lambda 清楚得多：

```
1 def sort_key(pair):
2     raw_value = pair[0]
3     number_value = self.parse_number(raw_value)
4     if number_value is not None:
5         return (0, number_value)
6     return (1, str(raw_value).lower())
7
8 items.sort(key=sort_key, reverse=reverse)
```

▲ `embedded_admin_tools/market_quote_window.py` · 表格点表头排序

这个 key 返回 `(0, 数字)` 或 `(1, 文本)`：**第一项是「能不能当数字看」的标志位**，于是所有数字行排在所有文本行前面，各自内部再比大小——表格排序里非常实用的一个小技巧。

多条件排序为什么不用连写多个 sort

Python 的 sort **稳定**，所以「先按 A 排、再按 B 排」也能得到正确结果，但那是写两次、遍历两遍。**用元组 key 一次搞定**，意图也更直白。



▲ 清单拖动重排进行中：被拖的那一行整体压暗，两行之间有一条 2px 插入指示线。松手之后算出的新顺序，最后就是写回 `todo_lists.sort_order` 的那一串值。

4.7 可变与不可变：一个必须记住的陷阱

这是 Python 最经典的坑之一，本项目在数据类那里正面撞上过：

```
1 # 错误写法：所有实例共用同一个列表！
2 @dataclass
3 class Bad:
4     tags: list[str] = []
5
6
7 # 正确写法：默认值由工厂函数「每次现造一个」
8 @dataclass
9 class TodoItem:
10     tags: list[str] = field(default_factory=list)
```

▲ `todo_db.py` · `TodoItem` 里的 `field(default_factory=list)`

根因不在 `dataclass`，在「默认值只求值一次」

函数（和方法）的默认值在定义时求值一次，之后所有调用共用同一个对象。所以 `def f(items=[])` 里的那一个空列表会被所有调用共享——往里 `append` 一次，下次调用还在。`@dataclass` 只是把这个行为原样继承了下来。判断规则很简单：默认值是可变对象（`list` / `dict` / `set`）就必须用工厂。`field(default_factory=list)` 的含义是「每次构造实例时调一次 `list()`」，于是每个实例都有自己的空列表。

不可变对象没有这个问题，所以 `TodoItem` 里几十个 `str` / `int` 字段都直接写默认值，只有 `tags` 需要 `default_factory`。

第五章 控制流

Python 的控制流语法极少，但有两种写法上的倾向值得学：**早退式（守卫子句）**与**扁平化嵌套**。项目里几乎所有稍长的函数都这么写。

5.1 if / elif / else 与条件表达式

```
1     if getattr(self, "_exiting", False):
2         return
3     try:
4         self._run_todo_alerts()
5     except Exception:
6         logger.exception("待办到点提醒巡检失败")
7     finally:
8         self.after(TODO_TICK_MS, self.todo_alert_check)
```

▲ main.py · todo_alert_check(): 先守卫、再干活、最后必续期

条件表达式（三目）是**表达式**不是语句，所以能塞进参数里：

```
1     mid_count = len(items) if self.current_scope == "completed" else len(open_items)
2     return items[:limit] if limit > 0 else items
3     "BODY" if heavy else "BODY_LIGHT"
```

▲ 三处真实用例 (todo_page.py / tools_launcher.py / ui_theme.py)

为什么 Python 没有 switch（以及后来为什么又有了）

老版本里用「if / elif 链」或「字典派发」代替。本项目 main.py 里按模块 key 切换到不同页面，用的就是字典 + 显式分支，而不是上百行 elif。3.10 起有了 match 语句，但它**不是 C 的 switch**：它是**结构匹配**，能匹配序列、映射、类实例，也支持 case _: 兜底。见 5.6。

5.2 守卫子句：把嵌套压平

所谓守卫子句，就是**先把所有「不满足就早点退出」的情况写在函数开头**，让真正的主逻辑留在最外层。对比一下：

```

1 # 不推荐：条件一层套一层
2 def export(items, path):
3     if items:
4         if path:
5             if len(items) > 0:
6                 # 真正的逻辑被推到第 3 层
7                 return write(items, path)
8
9
10 # 推荐：先排除掉不成立的情况
11 def export(items, path):
12     if not items:
13         return None
14     if not path:
15         return None
16     return write(items, path)

```

本项目的 `pending_alerts` 与 `parse_moment` 都是后者：

```

1 def parse_moment(value) -> Optional[datetime]:
2     """把 ``YYYY-MM-DD HH:MM`` 解析为 datetime；非法或空值返回 None。"""
3     parts = _norm(value).replace("T", " ").split(" ")
4     if len(parts) < 2:
5         return None
6     day = parse_day(parts[0])
7     clock = parts[1].split(":")
8     if day is None or len(clock) < 2:
9         return None
10    try:
11        hour, minute = int(clock[0]), int(clock[1][:2])
12    except ValueError:
13        return None
14    if not (0 <= hour <= 23 and 0 <= minute <= 59):
15        return None
16    return datetime(day.year, day.month, day.day, hour, minute)

```

▲ `todo_db.py · parse_moment()`：五道闸，一道比一道严

手工解析而不用 `strptime` 的原因

`datetime.strptime("2026-09-18 9:05", "%Y-%m-%d %H:%M")` 里 `%H` 期望两位小时，对 `9:05` 这种**非零填充**的写法在部分实现上解析不出来。手工 `int()` 拆没有这个不确定性，而且能顺手把「小时 25」这种荒谬值一并挡掉。当输入是「用户手填 / 别的系统塞进来的」时，宁可自己拆。

5.3 for 循环与它的好搭档

写法	用途
<code>for x in seq:</code>	直接遍历元素
<code>for i, x in enumerate(seq):</code>	同时要下标（可加 <code>start=1</code> ）

写法	用途
<code>for a, b in zip(xs, ys):</code>	并行遍历两个序列（长度不齐时以短的为准）
<code>for k, v in d.items():</code>	遍历字典
<code>for line in file:</code>	逐行读文件（不要 <code>readlines()</code> ）
<code>for i in range(n):</code>	按次数循环； <code>range(2, 10, 3)</code> 支持步长
<code>for x in reversed(seq):</code>	倒着来（不复制）

```

1     for column, ddl in (
2         ("alerted_for", "..."),
3         ("snooze_until", "..."),
4     ):
5         if column not in existing:
6             self.conn.execute(ddl)

```

▲ 遍历「二元组序列」时当场解包，是项目里最常用的循环形态

别在遍历列表时删除元素

`for x in items: items.remove(x)` 会**跳过元素**：删除会让后面的元素前移，而下标继续前进。正确做法是「遍历副本、删原表」（`for x in list(items): items.remove(x)`），或者直接重建（`items = [x for x in items if not 要删(x)]`）—— 后者更清楚。

5.4 while 与 break / continue

`while` 用于「不知道要循环几次」的场景。本项目里一个典型是日历推算 —— 从今天起一天天往前找，直到找到那个既不是周末、也不是法定假期的一天：

```

1     yield cur
2     cur += timedelta(days=1)

```

▲ `todo_db.py` · 工作日推算器（生成器，用 `yield` 逐个给出）

`break` 立即跳出循环，`continue` 跳到下一次迭代。两者的使用规矩是：`continue` 尽量放在循环体开头做守卫，让下半段保持平坦：

```

1     for row in rows:
2         item = TodoItem.from_row(row)
3         moment = alert_moment(item)
4         if moment is None or moment > now:
5             continue
6         text = moment_str(moment)
7         # .....下面才是真正要做的事

```

▲ `todo_db.py` · `pending_alerts()`：不合格的直接 `continue` 掉

for 的 else 分支

`for` 和 `while` 都可以带 `else`：循环自然结束时执行，被 `break` 打断时跳过。它解决的是「找了一圈没找到」这种情况，省掉一个标志位变量：

```
1 for item in items:
2     if item.id == target:
3         print("找到了")
4         break
5 else:
6     print("一圈下来没有")    # 只有没 break 过才走到这里
```

▲ 语法示例（本项目暂未使用；知道有这个写法就够）

5.5 海象运算符 := (3.8+)

它允许在表达式里赋值，主要价值是省掉一次重复计算。判断标准很明确：同一个函数调用要写两遍时才有必要用，否则只会让人多读两眼。

```
1 # 不用海象：匹配到要调用两次
2 m = pattern.search(text)
3 if m:
4     print(m.group(1))
5
6 # 用海象：调用一次
7 if (m := pattern.search(text)):
8     print(m.group(1))
```

▲ 典型用法；注意海象赋值处必须加括号

本项目基本没用到海象

因为项目的风格是「守卫子句 + 具名中间变量」——拆出来的名字本身就有可读性价值，而海象把赋值藏进条件里，反而更难扫读。知道它、但不滥用，是这一节的结论。

5.6 match / case (3.10+)

3.10 引入的 `match` 是结构模式匹配，比 `switch` 强得多：它可以按位置、按键、按类型去「拆」一个值，还支持带条件的 `case ... if ...`。

```
1 def describe(node):
2     match node:
3         case {"kind": "module", "key": key}:
4             return f"模块 {key}"
5         case {"kind": "group", "items": [first, *_]}:
6             return f"分组，第一个是 {first}"
7         case [x, y]:
8             return "二元组"
9         case _:
10            return "其他"
```

▲ 用项目里 `nav_sidebar.NAV_MODEL` 那种嵌套字典当输入会很自然（示例由本手册编写）

用不用它，先看运行环境

`match` 需要 **Python 3.10 以上**。本项目当前用 3.12 打包，语法上可用，但**没有引入**——因为 `match` 对这类「字典键判断」的场景并不比显式 `if node["kind"] == "module"` 更清楚，而且要照顾到「读代码的人可能不熟这个新语法」。能读懂新语法，和该不该在团队代码里用它，是两件事。

第六章 函数

函数是 Python 里唯一的抽象单位：它同时是「一段逻辑」和「一个对象」。GUI 编程之所以处处是函数，就是因为按钮、绑定、定时器全都要求「传一个可调用对象进来」。

6.1 定义、返回值、多返回值

没有 `return` 的函数返回 `None`。要返回多个值时，**返回的是元组**，调用方解包即可 —— 不需要任何特殊语法：

```
1 def _resolve_level(level):
2     """确定日志级别：显式参数 > 环境变量 > 默认 INFO。"""
3     if level is not None:
4         return level
5     raw = os.environ.get(ENV_LEVEL, "").strip().upper()
6     if raw:
7         value = getattr(logging, raw, None)
8         if isinstance(value, int):
9             return value
10    return DEFAULT_LEVEL
```

▲ `log_setup.py` · 三级优先级的兜底写法

注意这个函数的返回值**可能是调用方传进来的同一个对象**（`return level`）。这是有意的：不加工就直接原样传回去，把「判断」和「构造」分开。

6.2 参数的四种形态

形态	写法	说明
位置参数	<code>def f(a, b):</code>	按顺序传
默认参数	<code>def f(a, b=0):</code>	必须在位置参数之后
可变位置	<code>def f(*args):</code>	<code>args</code> 是元组
可变关键字	<code>def f(**kwargs):</code>	<code>kwargs</code> 是字典
仅关键字	<code>def f(a, *, b):</code>	* 之后的只能用 <code>b=</code> 传
仅位置	<code>def f(a, /, b):</code>	/ 之前的不能按关键字传 (3.8+)

仅关键字参数在本项目里用得很刻意。看这个界面组件的构造函数：

```
1 def __init__(self, master, *, bg: str, inner_bg: Optional[str] = None):
2     super().__init__(master, bg=bg)
3     self._bg = bg
```

▲ `todo_page.py` · `ScrollArea.__init__` 的 *

* 是在防止什么

`bg` 与 `inner_bg` 都是颜色字符串，位置传参时写反了是一个安静的 bug —— 界面底色和内容底色对调，程序照样跑，只是看起来怪。加上 `*` 之后，调用处必须写 `ScrollArea(parent, bg="#fff", inner_bg="#fafafa")`，两个颜色各自带着名字，顺序就再也不会搞错。

同理 `PendingAlerts` 那种「一次调三个开关」的函数更适合全部改成仅关键字。

6.3 可变默认参数的陷阱（第 4.7 节的正脸）

```
1 def add(item, bucket=[]):           # 危险：默认值只在定义时求值一次
2     bucket.append(item)
3     return bucket
4
5
6 add(1)                               # [1]
7 add(2)                               # [1, 2]  <- 上一次的 1 还在
```

▲ 反例（由本手册编写）

根因是「默认值只在定义时求值一次」

函数（和方法）的默认值在定义时求值一次，之后所有调用共用同一个对象。

所以上面那个空列表会被所有调用共享 —— 往里 `append` 一次，下次调用它还在。3.7 节讲的 `field(default_factory=list)` 是同一个坑的 `dataclass` 版本。

规矩：默认值一律用不可变对象；可变的一律用 `None` 哨兵或工厂函数。

```
1 def build_tree(nodes, seen=None):
2     seen = seen if seen is not None else set()    # 每次调用现造一个
3     ...
```

▲ 用 `None` 当哨兵的标准写法

6.4 作用域：LEGB 与闭包

Python 找名字的顺序是 **Local** → **Enclosing**（外层函数）→ **Global**（模块）→ **Builtin**（内建）。函数内部只能读外层的变量；要写，必须声明 `global`（写模块级）或 `nonlocal`（写外层函数级）。

```

1  _STRIKE_FONTS: dict = {}
2
3
4  def strike_font(base, widget):
5      """把元组字体转成带删除线的 ``tkinter.font.Font``。"""
6      key = tuple(base)
7      cached = _STRIKE_FONTS.get(key)      # 读全局字典：允许
8      if cached is not None:
9          try:
10             cached.actual()
11             return cached
12         except tk.TclError:
13             _STRIKE_FONTS.pop(key, None)  # 改全局字典的内容：也允许
14     font = tkfont.Font(root=widget, family=base[0], size=base[1],
15                        weight=("bold" if "bold" in styles else "normal"),
16                        overstrike=1)
17     _STRIKE_FONTS[key] = font            # 仍然是改内容，不是重新赋值
18     return font

```

▲ todo_page.py · 模块级缓存 + 函数读写标准姿势



▲ 完成态的两种标记同时生效：左侧勾选 + 标题文字中间一道删除线。这道横线只有 `overstrike=1` 的 `Font` 对象画得出来——上面那个字体缓存就是为它服务的。

「改字典内容」为什么不需要 `nonlocal` / `global`

规则是：**给名字重新赋值**才需要声明；`d[k] = v`、`d.pop(k)`、`lst.append(x)` 都不是重新赋值，它们只是在调用对象自己的方法。所以上面这段完全合法。如果写成 `_STRIKE_FONTS = {}`（重新绑定这个名字），函数里就必须先声明 `global`，否则 Python 会在函数内新建一个同名局部变量，外面那个纹丝不动——**这是最安静的一类 bug**。

闭包：函数记住了它出生时的环境

```

1  def make_strike_fontter(base):
2      """返回一个「把 base 变成带删除线字体」的函数。"""
3      cache = {}
4
5      def get(widget):
6          font = cache.get(widget)
7          if font is None:
8              font = tkfont.Font(root=widget, family=base[0], size=base[1],
9                               overstrike=1)
10             cache[widget] = font
11             return font
12
13     return get          # get 记住了 base 和 cache

```

▲ 闭包演示（由本手册编写，用于说明内层函数如何捕获外层变量）

```

1  for i in range(3):
2      buttons.append(lambda: print(i))    # 三个都会打印 2，不是 0 1 2

```

▲ 反例（由本手册编写）

闭包捕获的是「变量」而不是「当时的值」

三个 lambda 捕获的是**同一个变量 i**，循环结束时 i 已经是 2，所以点哪个按钮都打印 2。

两种修法：① 用默认参数把当前值**绑死在定义时**，写 `lambda i=i: print(i)`；② 抽出一个工厂函数，让 i 成为那次调用的局部变量。

这个坑在 GUI 里是高频雷区，因为「循环创建一排按钮」正是最常见的写法。

```

1      for node in nodes:
2          self._make_row(node, level=level, kind=kind)
3
4      def _make_row(self, node, *, level, kind):
5          """每行都由这个方法独立创建，node 是本次调用的局部变量。"""
6          row = tk.Frame(self)
7          row.bind("<Button-1>", lambda _e, n=node: self._on_click(n))

```

▲ nav_sidebar.py 的做法：用工厂方法（而非在循环里直接写 lambda）

上例中 `lambda _e, n=node: ...` 用了「默认参数绑定」这一招：`n=node` 在**定义时**就把当前的 node 存进了函数的默认值里，之后再怎么循环都与它无关。

6.5 lambda：只配做「一次性小函数」

lambda 的语法是 `lambda 参数: 表达式`，函数体必须是一个表达式，不能有语句、不能有 return、不能多行。它存在的意义只有一个：给 `key=`、`command=`、`bind()` 这类要「一个小函数」的地方当场塞一个。

```
1         self.bind("<Enter>", lambda e: self.set_hover(True))
2         self.bind("<Leave>", lambda e: self.set_hover(False))
3         if command is not None:
4             self.bind("<Button-1>", self._invoke)
```

▲ ui_components.py · 悬停进出用 lambda，真正的点击回调用具名方法

注意这段的分界很讲究：**纯转发一行调用的用 lambda，有实际逻辑的用方法**。lambda e: self.set_hover(True) 一眼就看完，而 _invoke 里要做状态判断、要触发回调、要返回值，所以它有名字。

场景	选择
key=lambda x: x[0]	lambda
command=lambda: self.do(1)	lambda
需要 try / 多行 / 分支	写具名函数，再传函数名
会被反复调用、需要单元测试	写具名函数
需要 __name__ 出现在日志里	写具名函数 (lambda 的名字是 <lambda>)

6.6 函数是对象：回调与高阶函数

这是理解 Tkinter 的关键：command=on_click 传的是函数对象本身，command=on_click() 传的是它的返回值——后者会在绑定时立刻调用一次，然后把返回值（通常是 None）当作回调。

```
1         dialog = TodoAlertDialog(
2             self, items, db=self.todo_db,
3             on_changed=self._after_todo_alert,      # 传函数对象，不加括号
4             on_open=self._open_todo_item,
5         )
```

▲ main.py · show_todo_alert() · 三个回调全是具名方法

常见的更高阶用法还有 functools.partial 与把函数放进容器里派发：

```
1  from functools import partial
2
3  # partial: 把部分参数先固定住，得到一个新的可调用对象
4  on_pick = partial(self.set_color, picker, hex_value)
5  on_pick()      # 等价于 self.set_color(picker, hex_value)
6
7  # 派发字典：按 key 找处理函数，比长串 if / elif 清楚
8  HANDLERS = {
9      "module_ops_expiry": self._build_expiry_page,
10     "module_life_todo": self._build_todo_page,
11 }
12 HANDLERS[key]()
```

▲ 由本手册编写；项目中等价的写法见 main.py 的页面切换

`__name__` 与 `functools.wraps`

函数对象自带 `__name__`、`__doc__`、`__module__`。写装饰器时如果不把这几个属性搬过去，被装饰的函数在日志里就会显示成包装函数的名字——本项目的 `logger.exception` 会带上模块名，所以装饰器一定要用 `functools.wraps`。见 6.7。

6.7 装饰器

装饰器就是「接收一个函数、返回一个新函数」的函数。`@deco` 写在 `def` 上方，等价于在这句 `def` 之后执行 `func = deco(func)`。

项目里在用的四种内建装饰器

装饰器	作用	项目里的位置
<code>@property</code>	把方法伪装成属性，读的时候不加括号	<code>TodoItem.due_day</code> 、 <code>ReminderSummary.has_items</code>
<code>@classmethod</code>	第一个参数是类本身（ <code>cls</code> ），常当「备用构造函数」	<code>TodoItem.from_row()</code> 、 <code>ToolboxDatabaseAdapter.from_source()</code>
<code>@staticmethod</code>	不接收 <code>self / cls</code> ，只是挂在类名下的普通函数	<code>todo_page.py</code> 里的三个纯函数
<code>@dataclass</code>	按注解自动生成 <code>__init__ / __repr__ / __eq__</code>	<code>TodoItem</code> 、 <code>TodoList</code> 、 <code>Typography</code> 、 <code>Palette</code>

```
1 @dataclass(frozen=True)
2 class Typography:
3     """字体 token（字号单位为 pt，10pt 约等于 13px）。
4
5     注意：Tk 只支持 normal / bold 两档字重（没有 500），
6     因此「只用两种字重」在这一层映射为 normal 与 bold。
7     """
8
9     hero: tuple = ("Microsoft YaHei UI", 18)
10    page_title: tuple = ("Microsoft YaHei UI", 16)
11    title: tuple = ("Microsoft YaHei UI", 11, "bold")
12    body: tuple = ("Microsoft YaHei UI", 10)
13    caption: tuple = ("Microsoft YaHei UI", 9)
```

▲ `ui_theme.py` · 用 frozen dataclass 当「常量表」

`frozen=True` 换来了什么

冻结之后实例属性不可再赋值，`Typography()` 本身又不可哈希（字段是 `tuple` 可以），关键是它向读代码的人宣告了「这是常量」。如果哪天有人写了 `TYPOGRAPHY.body = 12`，会立刻抛 `FrozenInstanceError`，而不是悄悄改坏全局样式。

自己写一个装饰器

```

1 import functools
2 import time
3
4
5 def timed(logger):
6     """记录被装饰函数的耗时。"""
7
8     def deco(func):
9         @functools.wraps(func)          # 保住 __name__ / __doc__
10        def wrapper(*args, **kwargs):
11            start = time.perf_counter()
12            try:
13                return func(*args, **kwargs)
14            finally:
15                cost = (time.perf_counter() - start) * 1000
16                logger.debug("%s 用了 %.1f ms", func.__name__, cost)
17
18        return wrapper
19
20    return deco
21
22
23 @timed(logger)
24 def load_workbook(path):
25     ...

```

▲ 带参数的装饰器：外三层、里一层

`functools.wraps` 不能省

少了它，被装饰的函数 `__name__` 会变成 `wrapper`、`__doc__` 会变成 `None`。后果是：日志里看到的全是 `wrapper`，`help()` 查不到文档，测试里按名字找函数也找不到。**所有手写装饰器都必须带 `functools.wraps`。**

```

1 @a
2 @b
3 def f(): ...
4
5 # 完全等价于
6 f = a(b(f))

```

▲ 多层装饰器

装饰器的执行顺序（从下往上）

离 `def` 最近的那个**先包上**，所以执行顺序是 `b` 先、`a` 后。记忆口诀是「**从上往下读，从下往上执行**」。

第七章 类与对象

本项目的界面全是类：每个页面一个 `class XxxPage(ttk.Frame)`，每个数据层一个 `class XxxDB`，每个独立窗口一个 `class XxxWindow`。这一章把类相关的语法一次讲清，并说明项目为什么这么组织。

7.1 class 与 self

```
1 class TodoDB:
2     """待办模块的数据层。
3
4     用法::
5
6         db = TodoDB(DATA_DIR / "expiry_manager.db")
7         lists = db.fetch_lists()
8         db.close()
9
10    多个页面共享同一个连接，所以**不要**在这里 commit 之后就 close。
11    """
12
13    def __init__(self, db_path: Path):
14        self.db_path = db_path
15        self.conn = sqlite3.connect(str(db_path), check_same_thread=False)
16        self.conn.row_factory = sqlite3.Row
17        self.conn.execute("PRAGMA foreign_keys = ON")
18        self._holidays_cache: Optional[dict[str, Holiday]] = None
19
20        self.create_tables()
21        self.seed_holidays()
22        self.seed_default_lists()
```

▲ todo_db.py · 节选（省略了 docstring 中的示例细节）

`self` 就是「这个实例本身」，必须显式写在参数表第一位。它不是关键字，只是一个约定俗成的名字——但请永远叫它 `self`，改成别的只会让人皱眉。

构造函数里这几行的顺序是有讲究的：**先建连接、先设 `row_factory`、再建表**。因为 `create_tables()` 内部要用 `self.conn`；而 `row_factory` 决定了后续所有查询返回的是 `sqlite3.Row` 还是元组——项目里所有 `row["name"]` 这种按列名取值，全靠它。

参数	含义	项目里的选择
<code>check_same_thread=False</code>	允许连接跨线程使用	定时器与子线程都要读库
<code>row_factory = sqlite3.Row</code>	查询结果可按列名取值	全项目通用写法
<code>PRAGMA foreign_keys = ON</code>	开启外键约束	SQLite 默认是关的， 必须显式开

7.2 实例属性 vs 类属性

写法	位置	存在哪	改动影响
<code>self.x = 1</code>	通常在 <code>__init__</code> 里	每个实例各一份	只影响该实例
<code>x = 1</code>	类体里（方法外）	所有实例共用一份	改类属性会影响所有实例

```
1 class TodoItem:
2     # 类属性：这是「字段的默认值」，dataclass 会据此生成 __init__
3     id: int = 0
4     title: str = ""
5     tags: list[str] = field(default_factory=list)
```

▲ `todo_db.py` · 带注解的类属性被 `@dataclass` 收编成实例属性

类属性如果是可变对象，就是第 4.7 节的坑

`tags: list = []` 看着像「每个实例一个空列表」，实际是**所有实例共用一个**。这也是为什么 `field(default_factory=list)` 必须写——`dataclass` 本身不会帮你把类属性拷贝成实例属性，它只是照抄默认值。

7.3 继承与 `super()`

本项目的继承几乎全部只有一层，而且**全是继承 GUI 框架的基类**：

```
1 class ScrollArea(tk.Frame):
2     def __init__(self, master, *, bg: str, inner_bg: Optional[str] = None):
3         super().__init__(master, bg=bg)    # 先让父类把控件建好
4         self._bg = bg
5
6         self.vsb = ttk.Scrollbar(self, orient="vertical")
7         self.vsb.pack(side="right", fill="y")
8
9         self.canvas = tk.Canvas(self, bg=bg, highlightthickness=0, bd=0,
10                                yscrollcommand=self.vsb.set)
11         self.canvas.pack(side="left", fill="both", expand=True)
12         self.vsb.configure(command=self.canvas.yview)
```

▲ `todo_page.py` · `ScrollArea`

`super().__init__()` **必须在最前面**

父类 `tk.Frame.__init__` 负责创建真正的控件对象。在它执行之前，`self` 还不是一个可用的控件——此时调用 `self.pack()` 或传 `self` 当父容器都会报 `TclError`。**规矩：子类 `__init__` 第一句永远是 `super().__init__(...)`。**

多层继承时 `super()` 会按 MRO（方法解析顺序）找下一个类。查看顺序用 `ClassName.__mro__`。本项目没用到菱形继承，所以 `super()` 的效果就是「父类」——**但仍然要用 `super()` 而不是 `tk.Frame.__init__(self, ...)`**，因为将来插入中间类时前者自动适配。

7.4 `property`：把方法当属性读

```

1  @dataclass
2  class TodoItem:
3      # -- 展示辅助 -----
4      @property
5      def due_day(self) -> Optional[date]:
6          """截止日期的 date 形式；未设置时返回 None。"""
7          return parse_day(self.due_date)
8
9      @property
10     def priority_mark(self) -> str:
11         return PRIORITY_MARKS.get(self.priority, "")
12
13     @property
14     def repeat_label(self) -> str:
15         return REPEAT_LABELS.get(self.repeat_rule, "永不")

```

▲ todo_db.py · TodoItem 的三个计算属性

调用处因此变得很干净：`item.repeat_label` 而不是 `item.repeat_label()`。判断该不该用 `property` 的标准是「它像不像一个属性」——「这条待办重复吗」是属性，「把这条待办推进到下一周期」是动作，后者必须是方法（项目里叫 `advance_repeat` 之类）。

只读 property 与可写 property

```

1  class Node:
2      def __init__(self):
3          self._active = False
4
5      @property
6      def is_active(self) -> bool:          # 读
7          return self._active
8
9      @is_active.setter
10     def is_active(self, value: bool):      # 写（名字必须是 <属性名>.setter）
11         self._active = bool(value)
12         self._paint()

```

▲ 由本手册编写；ui_components.py 里是把读写拆成 `is_active` 属性 + `set_active()` 方法

项目里为什么不给每个属性都配 setter

ui_components.py 里只定义了 `@property is_active`（只读），写用的是具名方法 `set_active(value)`。原因：**setter 会让「赋值」看起来是免费的，而这里赋值要触发重绘。**改成方法后，调用方一眼看得出「这一步有副作用」。凡是「写」会引发别的事情（重绘、落库、发通知）的地方，都不该用 `setter`。

7.5 classmethod 与 staticmethod

```
1 @dataclass
2 class TodoList:
3     id: int = 0
4     name: str = ""
5     color: str = DEFAULT_LIST_COLOR
6
7     @classmethod
8     def from_row(cls, row: sqlite3.Row) -> "TodoList":
9         return cls(
10             id=int(row["id"]),
11             name=_norm(row["name"]),
12             color=_norm(row["color"]) or DEFAULT_LIST_COLOR,
13             sort_order=int(row["sort_order"] or 0),
14         )
```

▲ todo_db.py · from_row() 是典型的「备用构造函数」

	第一个参数	调用方式	典型用途
普通方法	self	obj.m()	需要访问实例状态
@classmethod	cls	Cls.m() 或 obj.m()	备用构造函数；工厂
@staticmethod	无	Cls.m()	逻辑上属于这个类、但不需要实例

from_row 用 cls 而不是 TodoList 的原因

cls(...) 会在子类调用时自动构造子类实例。如果写死 TodoList(...)，将来继承出一个 TodoItemExt 再调 TodoItemExt.from_row(row)，拿到的却是一个 TodoList —— 这类 bug 非常难找。**工厂方法一律用 cls。**

```
1 class ToolboxDatabaseAdapter:
2     """系统工具箱数据库适配器，统一收口原始 sqlite 连接的传递方式。"""
3
4     def __init__(self, conn: sqlite3.Connection):
5         self._conn = conn
6
7     @classmethod
8     def from_source(cls, db_source):
9         if isinstance(db_source, cls):
10             return db_source # 已经是适配器，原样返回
11         if hasattr(db_source, "conn"):
12             return cls(db_source.conn) # 传进来的是上层 DB 对象
13         return cls(db_source) # 传进来的是裸连接
14
15     @property
16     def connection(self) -> sqlite3.Connection:
17         return self._conn
```

▲ tools_db.py · 兼容三种入参的适配器工厂

isinstance(db_source, cls) 这一句是**幂等**的关键：同一个适配器被传进去两次，第二次原样返回，不会套娃。

7.6 魔术方法：让自定义对象融入语言

魔术方法（dunder methods）是「用自定义类去接语言的操作符」的钩子。本项目实际用到的不多，但都用在了点子上：

魔术方法	什么时候被调用	项目里的位置
<code>__init__</code>	构造实例	随处可见
<code>__repr__</code>	<code>repr(obj)</code> 、调试器窗口、打印列表	<code>dataclass</code> 自动生成
<code>__eq__</code>	<code>==</code> 比较	<code>dataclass</code> 自动生成（按字段逐一比）
<code>__enter__</code> / <code>__exit__</code>	<code>with</code> 语句进入 / 退出	数据库连接的上下文管理
<code>__iter__</code>	<code>for x in obj</code>	工作日推算器
<code>__len__</code>	<code>len(obj)</code>	容器类组件
<code>__getitem__</code>	<code>obj[key]</code>	行数据包装

```
1 import sqlite3
2 from contextlib import contextmanager
3
4
5 @contextmanager
6 def connect(db_path):
7     """with 写法：无论正常结束还是抛异常，连接都会被关掉。"""
8     conn = sqlite3.connect(str(db_path))
9     conn.row_factory = sqlite3.Row
10    try:
11        yield conn
12    finally:
13        conn.close()
14
15
16 with connect(DATA_DIR / "expiry_manager.db") as conn:
17     rows = conn.execute("SELECT * FROM accounts").fetchall()
```

▲ 由本手册编写；项目里 todos 用的是「长连接 + 显式 close」，见下方说明

项目为什么没用 with 管连接

因为待办模块的连接是跨页面、跨定时器共享的长期连接（`todo_alert_check` 每 30 秒要用一次，提醒窗里点「完成」也要用）。每次操作都开关一次连接，在 SQLite 上意味着反复做文件级加锁，既慢又容易撞上 `database is locked`。

`with` 适合「用完即弃」的资源（临时文件、一次性查询），不适合长生命周期对象。判断标准是：这个资源的生命周期跟「一次操作」对齐，还是跟「整个程序」对齐。

`__repr__` 为什么值钱

`@dataclass` 会自动生成 `__repr__`，于是打印一条待办长这样：`TodoItem(id=7, title='交房租', due_date='2026-09-25', ...)`。调试时能直接看到字段名，比看 `(7, 3, '交房租', '')` 强太多——这是项目大量使用 `dataclass` 的直接收益之一。

7.7 dataclass：数据类的标准做法

```
1 @dataclass
2 class ReminderSummary:
3     """到期提醒摘要数据类。
4
5     属性:
6         overdue: 已过期资产列表
7         due_15:  15 天内到期资产列表（同结构）
8         due_30:  30 天内到期资产列表（同结构）
9
10    方法:
11        has_items -> bool  返回是否还有需要提醒的资产
12    """
13    overdue: list
14    due_15: list
15    due_30: list
16
17    @property
18    def has_items(self) -> bool:
19        return bool(self.overdue or self.due_15 or self.due_30)
```

▲ main.py · ReminderSummary

参数	作用
<code>@dataclass</code>	按注解生成 <code>__init__</code> / <code>__repr__</code> / <code>__eq__</code>
<code>frozen=True</code>	不可变；赋值抛 <code>FrozenInstanceError</code>
<code>eq=False</code>	不生成 <code>__eq__</code> （想用「同一性」比较时）
<code>order=True</code>	生成 <code><</code> <code>></code> 等比较方法（按字段顺序比）
<code>slots=True</code>	生成 <code>__slots__</code> ，省内存、防手滑加属性（3.10+）
<code>field(default=...)</code>	给字段加额外设置
<code>field(default_factory=list)</code>	可变默认值必须用它
<code>field(compare=False)</code>	参与比较时忽略该字段
<code>field(repr=False)</code>	打印时不显示（密码之类）

给 dataclass 加字段一定要带默认值

项目的 `TodoItem` 有 24 个字段，每个都有默认值。原因是**只要有一个字段没默认值，所有 `TodoItem(...)` 的调用点都必须补上它**——而调用点在数据层与界面层一共有几十处。带默认值之后，新增字段（`alerted_for`、`snooze_until` 就是这么加的）只需要动 `dataclass` 定义本身，其它代码一行都不用改。

代价是**字段顺序变得重要**：位置传参会按声明顺序对号入座。所以项目里所有 `TodoItem(...)` 都用关键字传参。

dataclass 与 NamedTuple、普通的类怎么选

数据为主、没有行为 → `@dataclass`。不可变、还要能当字典键 → `NamedTuple` 或 `frozen=True` 的 `dataclass`。有明显的状态与行为 → 普通类（比如 `TodoDB`、`ScrollArea`）。本项目的 `TodoDB` 虽然是「数据层」，但它是**行为的集合**，所以是普通类而不是 `dataclass`。

7.8 组合优于继承：本项目的分层就靠它

本项目只有「页面继承框架基类」这一种继承，其余关系全部是**组合**——对象把别的对象当零件用：

```
1 ExpiryManagerApp (主窗口)
2   |—— self.todo_db      = TodoDB(...)          # 数据
3   |—— self.todo_view    = TodoPage(...)        # 界面
4   |—— self.tray         = TrayController(...)  # 托盘
5   |—— self.todo_view 也持有 self.todo_db (注入)
```

▲ 组合关系（真实字段名以 `main.py` 为准）

组合的好处在这里很具体：`TodoPage` 只认「一个长得像 `TodoDB` 的东西」，所以测试时塞一个替身进去就能跑界面测试，完全不需要真数据库。如果用继承（`class TodoPage(TodoDB)`），页面和数据就焊死了。

拿不准时的判断标准

问一句「**A 是不是一种 B**」。 `TodoPage` 是一种 `tk.Frame` 吗？是——用继承。 `TodoPage` 是一种 `TodoDB` 吗？不是，它是**用一个 `TodoDB`**——用组合。这条「is-a / has-a」的判断在 90% 的情况下够用。

第八章 异常处理

异常不是错误，是**控制流的一种**。Python 的哲学是「先让它抛出来，在知道怎么处理的地方处理掉」，而不是层层返回错误码。但「在哪里处理」和「处理成什么样」是本项目花了不少笔墨的地方。

8.1 try 语句的四个部分

部分	执行时机	用途
<code>try:</code>	总是	放可能抛异常的代码
<code>except E:</code>	try 里抛出 E 时	处理；可写多个， 从上往下匹配
<code>else:</code>	try 里没抛异常时	放「只在成功后才做」的事，别塞进 try
<code>finally:</code>	无论如何	清理：关文件、恢复状态、续期定时器

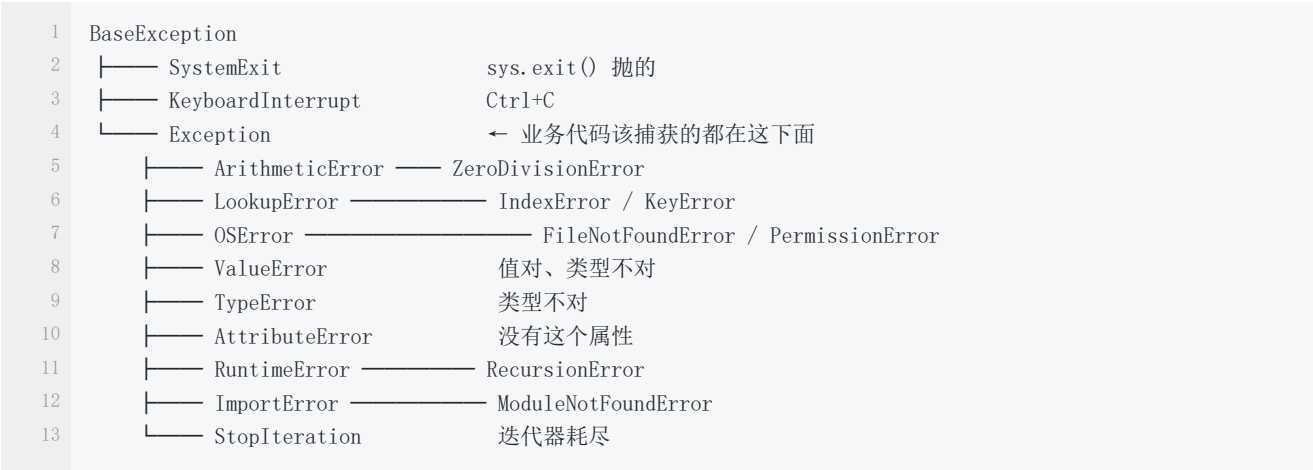
```
1 def todo_alert_check(self):
2     """巡检一次「有没有提醒该响了」。"""
3     if getattr(self, "_exiting", False):
4         return
5     try:
6         self._run_todo_alerts()
7     except Exception:
8         logger.exception("待办到点提醒巡检失败")
9     finally:
10        self.after(TODO_TICK_MS, self.todo_alert_check)
```

▲ main.py · 一个把四个部分用到极致的例子（这里只用了三个）

为什么续期放在 finally 而不是 try 的末尾
这是本项目的定时器纪律：**「续期」必须无条件执行**。
如果写成 `try: 干活(); self.after(...)` —— 一旦「干活」抛异常，`after` 永远不会被调用，**这个定时器就彻底死了**，之后再也不会有任何提醒。这类 bug 用户不会看到报错，只会觉得「提醒时好时坏」。测试里专门塞了一个会抛异常的替身来钉住这条。

`except Exception` **而不是裸** `except:`
裸 `except:` 会连 `KeyboardInterrupt` (Ctrl+C) 和 `SystemExit` 一起吞掉，让程序无法正常中断。项目里所有兜底都是 `except Exception`，只有极少数「清理路径」才用裸 `except` 且立刻继续抛。

8.2 常见内置异常与层级



▲ 只需要记住三件事：业务兜底捕 Exception、LookupError 管下标和键、其余按需

异常	典型触发	本项目里的位置
ValueError	int("abc")、strptime 格式不符	parse_moment() 拆时间
KeyError	d["不存在的键"]	优先用 d.get(k, 默认) 规避
TypeError	1 + "a"、参数个数不对	编码 bug，不该捕获
AttributeError	None.foo、访问不存在的方法	import pygments 之后取属性
OSError / FileNotFoundError	文件不存在、权限不足	日志目录建不出来时降级
sqlite3.Error	SQL 错误、表不存在	数据层
tk.TclError	控件已销毁、字体无效	界面层点击态与字体缓存
ImportError	可选依赖没装	openpyxl 的延迟导入

8.3 raise 与异常链 from

```

1  def load_excel_assets(file_path: Path) -> list[dict]:
2      try:
3          from openpyxl import load_workbook
4      except ImportError as exc:
5          raise RuntimeError("缺少 openpyxl, 请先执行 pip install -r requirements.txt") from exc
```

▲ main.py · 把「依赖没装」翻译成「用户看得懂的一句话」

raise ... from exc 保留了**原始异常**，Python 会把两条都打印出来：

```
1 Traceback (most recent call last):
2   File "main.py", line 2323, in load_excel_assets
3     from openpyxl import load_workbook
4 ModuleNotFoundError: No module named 'openpyxl'
5
6 The above exception was the direct cause of the following exception:
7
8 RuntimeError: 缺少 openpyxl, 请先执行 pip install -r requirements.txt
```

▲ from 会把「直接原因」写在上面，排查时两层都能看到

写法	效果
<code>raise</code>	在 <code>except</code> 里原样重抛，保持原异常
<code>raise NewError(...)</code>	抛新的，原异常挂成 <code>__context__</code> （隐式链）
<code>raise NewError(...) from exc</code>	显式链 ，明确「是它导致的」
<code>raise NewError(...) from None</code>	掐断链 ，只显示新异常（当原异常是噪音时）

什么时候该 `from None`

当原始异常对使用者毫无意义、只会吓人时 —— 比如「配置文件里某个字段格式不对」，底层抛的是 `KeyError: 'port'`，对用户没用。这时候 `from None` 只留自己那句。但**调试期不要掐** —— 排障时你正需要那层原始信息。

8.4 两种处理风格，以及怎么选

本项目里两种风格都在用，而且分得很清楚：**记日志** vs **静默降级**。

风格一：记日志（出问题了我要知道）

```
1 def _after_todo_alert(self):
2     """提醒窗里点过「完成」/「稍后提醒」之后刷新待办页。"""
3     try:
4         self.todo_view.refresh()
5     except Exception:
6         logger.exception("提醒处理后刷新待办页失败")
```

▲ `main.py · logger.exception` 会自动带上完整堆栈

`logger.exception` 等价于 `logger.error(..., exc_info=True)`，只能在 `except` 块里用（它靠 `sys.exc_info()` 取当前异常）。它的输出会落到 `ExpiryManager_Data/logs/app.log`。

风格二：静默降级（失败也无关紧要）

```
1 import pygments
2
3
4 def _get_highlighter():
5     try:
6         from pygments.lexers import PythonLexer
7         from pygments.formatters import HtmlFormatter
8     except ImportError:
9         return None          # 没装 pygments 就没有语法高亮，功能照常
10    return PythonLexer(), HtmlFormatter()
```

▲ 可选的第三方依赖：装不装都能跑

场景	选择	理由
核心流程失败（存库、读配置）	记日志 + 上报用户	静默失败会让数据悄悄丢失
可选依赖缺失	静默降级	功能变弱，但程序可用
清理路径（关连接、恢复界面）	静默忽略	此刻再抛异常只会盖住真正的错
定时器 / 事件回调	记日志 + 保证继续运行	断掉之后不会自己恢复
用户输入格式错	不抛异常，返回 None / 默认值	属于正常业务分支

静默降级最大的风险是「假装没事」

`except: pass` 会让一个功能长期不工作而没人发现。本项目因此定了一条：静默降级必须写明「降级之后是什么状态」。上面那段 `pygments` 的例子，`return None` 的语义是「没有高亮器」，调用方拿到 `None` 会走纯文本渲染——这就是一句能读懂的空兜底，而不是「不知道发生了什么，反正没崩」。

8.5 assert：只用来钉「不可能发生」

assert 会在 -O 优化模式下被整体删掉

`python -O script.py` 时所有 `assert` 语句消失。所以：
绝不能用 `assert` 做输入校验或安全检查（`assert user.is_admin` 这种写法在 `-O` 下直接失效）。
该用的是「内部不变量」——描述「如果这行不成立，说明代码本身写错了」。

```
1 # 内部不变量：锚点必须命中一次，否则说明源码已经被改动过
2 assert raw.count(old) == 1, f"锚点命中 {raw.count(old)} 次，本应恰好 1 次"
```

▲ 本项目所有字节级补丁脚本的第一道闸

项目里 `assert` 出现最多的地方是两个：补丁脚本（保证替换唯一）与回归测试（断言行为）。业务代码里几乎见不到——业务条件一律用 `if` 加正常控制流处理。

8.6 自定义异常

```
1 class CaptureFailed(RuntimeError):
2     """抓图失败（窗口句柄不对、窗口已销毁、或抓到的是纯色空图）。"""
```

▲ scripts/window_shot.py · 一个自定义异常的完整定义只要三行

继承哪个基类是有讲究的：这里选 `RuntimeError` 而不是 `Exception`，含义是「运行环境的问题，不是调用方传错了参数」——后者该继承 `ValueError`，参数类型问题该继承 `TypeError`。选对了，调用方就能写出有意义的 `except`。

```
1 colors = image.getcolors(maxcolors=1 << 20) or []
2 if len(colors) < min_colors:
3     raise CaptureFailed(
4         f"只抓到 {len(colors)} 种颜色（{colors[:1]}）—— PrintWindow 没生效")
```

▲ 抛的时候带上现场数据，收到的人不用再复现一次

自定义异常的规矩

- ① **定义要短**，docstring 说清「什么情况下会抛」；
- ② **基类选对** (`ValueError` / `TypeError` / `RuntimeError` / `OSError`)，让调用方能按语义捕获；
- ③ **消息里带上下文**（期望值、实际值、关键变量），不要只写「失败了」；
- ④ 不要为了「显得专业」而给每个函数配一个异常类——本项目的 83 个文件里只定义了一个。

第九章 模块、包与导入

一个 `.py` 文件就是一个模块，一个含 `__init__.py` 的目录就是一个包。本项目的导入语句里藏着几条实用经验，这一章一起讲。

9.1 导入的四种写法

写法	含义	项目里的例子
<code>import sqlite3</code>	导入模块，用时 <code>sqlite3.Row</code>	标准库一律这么导
<code>from pathlib import Path</code>	导入名字	文件路径操作
<code>from datetime import date, datetime, timedelta</code>	导入多个名字	时间计算
<code>import numpy as np</code>	起别名	本项目没用到，但 <code>import tkinter as tk</code> 同理

```
1 from __future__ import annotations
2
3 import calendar
4 import json
5 import sqlite3
6 from dataclasses import dataclass, field
7 from datetime import date, datetime, timedelta
8 from pathlib import Path
9 from typing import Optional
```

▲ `todo_db.py` 的导入区 —— 顺序是「标准库 → 第三方 → 本项目」

导入顺序不是形式主义

PEP 8 建议分三段（标准库 / 第三方 / 本项目），段间空一行。好处很实际：**一眼看出这个模块依赖了几个外部包**。本项目的第三方依赖只有 `openpyxl` 与可选的 `pygments`，所以「第三方段」经常是空的 —— 这本身就是一条信息：**这个程序装起来几乎没有外部依赖**。

`from __future__ import annotations` 必须放最前，它是编译器指令，位置错了会直接 `SyntaxError`。

别用 `from module import *`

```
1 # 不推荐：不知道导进来了什么，还可能覆盖同名变量
2 from todo_db import *
3
4
5 # 推荐：明确列出
6 from todo_db import TodoDB, TodoItem, ALERT_GRACE_MINUTES
```

▲ 对比（项目没有使用星号导入）

9.2 if `__name__` == "`__main__`"

`__name__` 是模块的「名字」：被导入时是模块名，被直接运行时是字符串 "`__main__`"。所以这个守卫的含义是「只有当我被当作脚本运行时才执行」。

```

1  if __name__ == "__main__":
2      sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
3      import tkinter as tk
4
5      root = tk.Tk()
6      root.title("截图自检")
7      root.geometry("320x140+80+80")
8      tk.Label(root, text="PrintWindow 自检", font=("Microsoft YaHei UI", 14)).pack(pady=30)
9      root.update()
10     try:
11         image = shot_widget(root, out)
12         print(f"OK 抓到 {image.size[0]}x{image.size[1]} → {out}")
13     finally:
14         root.destroy()

```

▲ `scripts/window_shot.py` 末尾：既是库，也是可自检的命令行工具

本项目的 `scripts/` 下几乎每个文件都有这一段，好处是同一个文件既能被测试导入，也能单独跑一次做自检——`python scripts/window_shot.py` 会立刻告诉你截图功能在这台机器上可不可用。

守卫里做的事要「无副作用」或明说副作用

`main.py` 是唯一一个没有 `if __name__ == "__main__"` 守卫的主模块吗？并不是——它也有，而且里面只做一件事：建单实例互斥、配日志、起 `mainloop()`。千万不要在模块顶层写有副作用的代码（建文件、连数据库、弹窗），否则别人只是 `import` 一下就触发了。

9.3 包、相对导入与 `__init__.py`

```

1  embedded_admin_tools/
2  |—— __init__.py
3  |—— api_demo_window.py      界面
4  |—— crypto_window.py
5  |—— returnCode.json        数据
6  |—— services/              业务与数据
7      |—— api_client.py
8      |—— crypto_service.py
9      |—— error_code_service.py
10     |—— icon_service.py
11     |—— interface_service.py

```

▲ 内置工具包的分层

```

1 import json
2 import tkinter as tk
3
4 from .services.api_client import ApiClient
5 from .services.crypto_service import DESCryptoService
6 from .services.error_code_service import ErrorCodeService
7 from .services.icon_service import save_temp_icon

```

▲ `embedded_admin_tools/market_quote_window.py` · 用相对导入引同包内的模块

写法	含义
<code>from . import x</code>	同包内
<code>from .sibling import x</code>	同包的子模块
<code>from .. import x</code>	上一级包
<code>from .services.x import Y</code>	同包的子包

`__init__.py` 现在可以省略，但本项目仍然放着

Python 3.3+ 支持「命名空间包」，没有 `__init__.py` 也能导入。但显式放着它有两个好处：① **让工具（打包器、IDE）明确知道这是包**；② 它是「包级初始化」的落点。本项目选择保持显式。

另一个值得记的细节：`embedded_admin_tools/` 根目录下有若干同名文件（如 `error_code_service.py`），它们是带 **docstring** 的 6 行兼容包装，内部转发到 `services/` 里的真实现 —— 为的是老的 `import` 路径不断。看起来像重复代码，其实是有意的兼容层，不要删。

9.4 延迟导入：把 800 毫秒挪出启动路径

```

1 def load_excel_assets(file_path: Path) -> list[dict]:
2     try:
3         from openpyxl import load_workbook      # 放在函数里，不放在文件顶部
4     except ImportError as exc:
5         raise RuntimeError("缺少 openpyxl，请先执行 pip install -r requirements.txt") from exc

```

▲ `main.py` · `openpyxl` 按需导入

为什么不用模块级 import

`openpyxl` 的导入要花大约 **800 毫秒** —— 对一个双击就期待看到界面的桌面程序来说，这是能感觉到的延迟。而「导入 Excel」是低频操作，很可能一次都用不上。

判断标准：贵 + 少用 = 延迟导入；便宜 + 常用 = 放文件顶部。延迟导入的代价是「每次调用都要查一次 `sys.modules`」（微秒级，可忽略），以及错误发现得晚（所以要配一个清楚的 `ImportError` 提示）。

延迟导入不能用在「热路径」里

如果一个函数每秒被调用几百次（比如渲染循环、事件处理），在里面写 `from x import y` 虽然 Python 有缓存不会重新加载，但每次仍要走一遍查找与加锁逻辑。**贵的东西放函数里，热的路径别放。**

9.5 冻结（打包）后的路径

```
1 def get_base_dir() -> Path:
2     if getattr(sys, "frozen", False):
3         return Path(sys.executable).resolve().parent
4     return Path(__file__).resolve().parent
5
6
7 BASE_DIR = get_base_dir()
8 DATA_DIR = BASE_DIR / "ExpiryManager_Data"
```

▲ main.py · 一个函数解决「源码运行」与「打包运行」的路径差异

	源码运行	PyInstaller 打包后
<code>__file__</code>	.py 文件的路径	指向临时解包目录（且每次运行都不同）
<code>sys.executable</code>	python.exe 的路径	exe 自身的路径
<code>sys.frozen</code>	不存在	<code>True</code>
<code>sys._MEIPASS</code>	不存在	临时解包目录（放只读资源）

这是打包程序最容易踩的坑

把数据目录建在 `__file__` 旁边，在打包版里会建到临时解包目录——**程序关掉数据就没了**，而且每次运行都从空白开始。正确做法就是上面这三行：**判断 `sys.frozen`，用 `sys.executable` 定位 exe，把数据放在 exe 同级目录**。顺带一个排查技巧：`sys._MEIPASS` 存在就说明正在打包环境里跑，打印它可以确认资源被解包到哪了。

9.6 sys.path 与脚本运行

```
1 if __name__ == "__main__":
2     sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
3     import tkinter as tk
```

▲ scripts/window_shot.py：把项目根目录加进搜索路径，才能 import 项目模块

运行的脚本所在目录会自动进 `sys.path`，但**脚本的父目录不会**。所以scripts/ 下的工具要导入根目录的 `todo_db`，就得自己插一条路径（`parent.parent` 就是根目录）。

sys.path.insert(0, ...) 里的 0

0 表示插到**最前面**，优先级最高。用 `append` 是插到最后，可能被同名的已安装包抢先。这里要的是「优先用项目里的这一份」，所以是 0。

第十章 类型注解

类型注解是给「读代码的人」和「静态检查工具」看的。**运行时它默认不生效**——写成 `def f(x: int)` 之后传字符串照样能跑。理解这一点，就不会把它当成语言层面的类型约束了。

10.1 注解写在三个地方

```
1 count: int = 0 # 1. 变量
2 cache: dict = {} # 有注解就一定会成为 dataclass 字段
3 # (所以类体里给变量加注解要谨慎)
4
5 def snooze(self, item_id: int, # 2. 参数
6     minutes: int = SNOOZE_MINUTES,
7     now: Optional[datetime] = None) -> str: # 3. 返回值
8     ...
```

▲ 三种位置的注解 (todo_db.py · snooze 的签名)

类体里的注解有额外含义

在 `@dataclass` 装饰的类里，**每一条带注解的类属性都会变成一个字段**。所以下面这两种写法语义完全不同：
`x: int = 0` → `dataclass` 认它是字段，进 `__init__` 参数表。
`x = 0` → 只是普通类属性，**不进** `__init__`。
想加一个「不属于构造参数」的类级常量，就不要写注解，或者用 `field(init=False)`。

10.2 Optional 与联合类型

写法	含义	备注
<code>Optional[datetime]</code>	<code>datetime</code> 或 <code>None</code>	等价于 <code>Union[datetime, None]</code>
<code>Union[int, str]</code>	两者之一	3.10 起可写 <code>int str</code>
<code>int None</code>	同上 (3.10+)	旧版本解释器会直接 SyntaxError
<code>Any</code>	任意类型	等于放弃检查，少用
<code>Literal["a", "b"]</code>	限定几个字面量值	很适合表示「状态字符串」
<code>Final</code>	常量标记	<code>Final[int] = 10</code>

```
1 from typing import Optional
2
3 def alert_moment(item: "TodoItem") -> Optional[datetime]:
4     """一条待办「下一次该响」的时刻；没设日期或时间时返回 None。"""
5     if not _norm(item.due_date) or not _norm(item.due_time):
6         return None
7     ...
```

▲ `todo_db.py` · `Optional[...]` 是项目里最常用的类型包装

本项目写 `Optional[X]` 而不是 `X | None`

两种写法在 3.10+ 等价，而且加了 `from __future__ import annotations` 之后连 3.7 都能用 `|` 写法（因为注解不再被求值）。本项目仍统一用 `Optional[X]`，原因是**它也是给不熟悉新语法的读者看的**，而且项目里有一部分模块没有加 `future` 导入。**在同一个代码库里保持一致，比「用上最新语法」更重要。**

10.3 容器泛型

```
1 REPEAT_RULES: list[tuple[str, str]] = [("none", "永不"), ...]
2 LIST_ICONS: list[str] = ["list", "check", ...]
3 moments: dict[int, str] = {}
4 counts: dict[str, int] = {}
5 self._holidays_cache: Optional[dict[str, Holiday]] = None
```

▲ `todo_db.py` · 泛型容器的几种常见嵌套

写法	含义
<code>list[str]</code>	字符串列表
<code>tuple[str, str]</code>	固定两个元素的元组
<code>dict[str, int]</code>	键是 <code>str</code> 、值是 <code>int</code>
<code>list[tuple[str, str]]</code>	二元组构成的列表（本项目的表数据都是这个）
<code>set[int]</code>	整数集合
<code>Sequence[str]</code>	只读序列（比 <code>list</code> 更宽松，参数建议用它）
<code>Iterable[str]</code>	可迭代对象
<code>Iterator[str]</code>	迭代器

注解里的泛型不参与运行

`list[str]` 在运行期并不检查元素类型，它是「给人看的契约」。真要在运行期校验得自己写断言，或者用 `pydantic` 之类的库。本项目选择**不校验**——数据来源是自己写的 SQL 与自己的界面，注解的作用是「读代码时不用猜」。

10.4 Callable：把函数当参数传时怎么标注

```
1 from typing import Callable, Optional
2
3
4 class ConsolePage(ttk.Frame):
5     def __init__(self, master, log_path: Path,
6                   on_status: Optional[Callable[[str], None]] = None):
7         ...
```

▲ console_page.py · 回调参数的标注方式

写法	含义
<code>Callable[[str], None]</code>	收一个 str、返回 None 的函数
<code>Callable[..., None]</code>	参数随便、返回 None
<code>Callable[[], None]</code>	无参数、无返回值 (<code>command=</code> 最常见)
<code>Optional[Callable[[int], None]]</code>	可为 None 的回调 —— 传不传都行

`Optional[Callable[...]]` 是界面代码里的标准形状

组件的回调参数几乎总是「可选」的 —— 谁用谁传。项目里所有窗口类的构造参数都遵循这个形状：
`on_changed= / on_open= / on_close=`，默认 `None`，用时先判 `if self.on_open:` 再调用。

```
1         self._on_open = on_open
2         ...
3     def _open(self, item):
4         if self._on_open is None:
5             return
6         self._on_open(item.id)
```

▲ 回调的调用点必须判空（本项目所有组件都遵守）

10.5 from __future__ import annotations

```
1 from __future__ import annotations    # 必须放在文件第一条语句之后
2
3 class TreeNode:
4     def attach(self, parent: TreeNode) -> TreeNode:    # 不用写引号了
5         ...
```

▲ 加了它之后，注解不必是「运行时可求值」的

	不加 future	加了 future
注解求值时机	定义时 立即求值	全部变成字符串，永不求值
前向引用	必须写 <code>-> "TodoList"</code>	直接写 <code>-> TodoList</code>
用新语法 (<code>int str</code>)	要 3.10+	3.7+ 都行

	不加 future	加了 future
性能	每个函数定义都要求值注解	零开销
用 <code>get_type_hints()</code>	直接可用	仍可用（会主动求值字符串）

```
1 @dataclass
2 class TodoList:
3     @classmethod
4     def from_row(cls, row: sqlite3.Row) -> "TodoList":    # 没有 future 导入时的写法
5         return cls(...)
```

▲ `todo_db.py` 用了引号包住类名 —— 因为它有 `future` 导入之前的历史写法

引号包类型名是「没有 `future` 导入」时的必需动作

-> `TodoList` 写在 `class TodoList` 内部时，类本身还没定义完，立即求值会抛 `NameError`。加引号让它延迟成字符串即可。**加了 `from __future__ import annotations` 之后就不需要引号了** —— 本项目两种写法并存（老模块带引号、新模块带 `future`），新写代码建议统一带 `future`。

10.6 注解不参与运行 —— 一条必须记住的边界

```
1 >>> def f(x: int) -> str:
2     ...     return x          # 返回的是 int，不是 str
3 >>> f("我甚至传了个字符串")
4 '我甚至传了个字符串'        # 一切正常，没有任何报错
```

▲ 注解完全不影响运行

想要运行期检查，有三条路：**手动 `isinstance`、用 `pydantic` / `attrs` 这类校验库、或者靠测试**。本项目走第三条 —— 1053 个断言构成的回归测试就是它的「类型检查器」，而且是**检查行为而不是检查声明**的那种。

类型注解在本项目的实际定位

它不是约束，而是**文档**。价值体现在三处：① IDE 的自动补全与跳转；② 读代码时不用翻实现就能知道参数形状；③ `@dataclass` 靠它生成 `__init__`（这是唯一「有实际作用」的用法）。

所以项目里**没注解的地方也不必补** —— 补了不影响运行，漏了不出错。但 `@dataclass` 的字段必须有注解，那是硬要求。

第十一章 标准库实战

Python 的「自带电池」在这项目里用得很实际：**没有引入任何 Web 框架、ORM、配置库**，全部靠标准库完成。这一章按模块讲，每个都配项目里的真实用法。

11.1 pathlib：路径就用 / 拼

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent
4 DATA_DIR = BASE_DIR / "ExpiryManager_Data"
5 LOG_DIR = DATA_DIR / "logs"
6 DB_PATH = DATA_DIR / "expiry_manager.db"
7
8 LOG_DIR.mkdir(parents=True, exist_ok=True)      # 不存在就建，存在也不报错
```

▲ main.py · 路径拼接与目录创建

操作	写法
拼接	p / "sub" / "file.txt" (/ 被重载成拼接)
取父目录	p.parent、p.parents[1]
取文件名 / 后缀 / 主干	p.name、p.suffix、p.stem
绝对化	p.resolve()
存在性	p.exists()、p.is_file()、p.is_dir()
建目录	p.mkdir(parents=True, exist_ok=True)
遍历	p.iterdir()、p.glob("*.py")、p.rglob("*.py")
读 / 写文本	p.read_text(encoding="utf-8") / p.write_text(...)
读 / 写字节	p.read_bytes() / p.write_bytes(...)

`write_text` 会把行尾换成 CRLF (Windows)

`Path.write_text()` 走的是文本模式，Windows 下默认把 `\n` 翻译成 `\r\n`。本项目因此踩过一次：一个纯 LF 的测试脚本被 `write_text` 写回后变成 946 个 CRLF，`git diff` 从「改几行」炸成「改 946 行」。要精确保住行尾，只能用 `write_bytes()`。详见 13.1、13.2。

11.2 datetime 与 calendar：日期计算的三个必备套路

套路一：按自然月推进，天数不足时收敛到月末

```
1 def add_months(base: date, months: int) -> date:
2     """按自然月推进，遇到目标月天数不足时收敛到该月最后一天。
3
4     例：1 月 31 日 + 1 个月 → 2 月 28/29 日（而不是抛异常或滚到 3 月）。
5     """
6     total = (base.year * 12 + base.month - 1) + months
7     year, month = divmod(total, 12)
8     month += 1
9     day = min(base.day, calendar.monthrange(year, month)[1])
10    return date(year, month, day)
11
12
13 def add_years(base: date, years: int) -> date:
14     """按自然年推进，2 月 29 日在平年收敛到 2 月 28 日。"""
15     year = base.year + years
16     day = min(base.day, calendar.monthrange(year, base.month)[1])
17     return date(year, base.month, day)
```

▲ todo_db.py · 重复规则里「每月 / 每年」的推进逻辑

这里的三个知识点

- ① `divmod(总月数, 12)` 一次同时拿到年和月 —— 把「年 + 月」压成一个线性坐标再除，比手工处理进位干净得多。
- ② `calendar.monthrange(y, m)[1]` 拿到该月天数，免去自己写闰年判断。
- ③ `min(base.day, 月末)` 是「收敛」而非「报错」：1 月 31 日加一个月落到 2 月 28/29 日，而不是抛 `ValueError`。**日期推进的默认策略应该是收敛** —— 用户设了「每月 31 日」，到 2 月时他期待的是「2 月最后一天」，不是一个报错。

套路二：格式化与解析要配对

```
1 def moment_str(value: datetime) -> str:
2     """datetime → ``YYYY-MM-DD HH:MM``（提醒精确到分钟，秒没有意义）。"""
3     return value.strftime("%Y-%m-%d %H:%M")
4
5
6 def _now() -> str:
7     """当前时间的 ISO 字符串（精确到秒）。"""
8     return datetime.now().isoformat(timespec="seconds")
```

▲ todo_db.py · 项目里时间入库只走这两个函数

格式码	含义	易错点
<code>%Y-%m-%d</code>	2026-09-21	大写的 <code>%Y</code> 是四位年
<code>%H:%M</code>	14:05	24 小时制用大写 <code>%H</code> ； <code>%I</code> 是 12 小时制
<code>%S</code>	秒	
<code>%f</code>	微秒	6 位

格式码	含义	易错点
%j	一年中的第几天	
%a / %A	星期缩写 / 全称	受系统语言影响，别用来判断星期

中文 Windows 上 %a 出来是中文星期

`strftime("%a")` 依赖系统 locale，在中文系统上可能是「周一」。要判断星期几一律用 `date.weekday()` (**0 = 周一**) 或 `isoweekday()` (**1 = 周一**)，不要解析格式化的字符串。本项目判断「是不是周末」用的就是数值。

套路三：比较与差值

```

1 now = now or datetime.now()
2 base = alert_moment(item) or now
3 nxt = max(base, now) + timedelta(minutes=max(1, minutes))

```

▲ `todo_db.py · snooze()`: `max(datetime, datetime)` 也是一行

`datetime` 支持比较、支持加减 `timedelta`，所以「两个时间取较晚的那个」直接用内建 `max` 就行，不需要写 `if`。

`datetime.now()` 是「本机时间」，没有时区信息

它返回的是 naive datetime (`tzinfo` 为 `None`)，含义是「这台机器认为的本地时间」。本项目是**单机自用工具**，所有时间都取自本机、也只在本机比较，所以 naive 时间没有问题。

但只要出现「跨机器同步」「服务器时间」「夏令时」任何一项，就必须换成 `datetime.now(timezone.utc)` 存 UTC、展示时再 `astimezone()` 到本地——否则一定会在某个时刻错一小时。

11.3 json：结构化数据的通用交换格式

```

1 @classmethod
2 def from_row(cls, row: sqlite3.Row) -> "TodoItem":
3     raw = row["tags"]
4     if isinstance(raw, str):
5         try:
6             tags = json.loads(raw)
7         except Exception:
8             tags = [t.strip() for t in raw.split(",") if t.strip()]
9     elif isinstance(raw, list):
10         tags = raw
11     else:
12         tags = []

```

▲ `todo_db.py · 从数据库读 JSON 列，带三级降级`

这段是本项目里「**宽容解析**」的范本：先按 JSON 解；解不出来就退化成逗号分隔的纯文本；连类型都不对（既不是字符串也不是列表）就当空。**老旧数据、手工改过的数据都能读进来**，不会因为一行脏数据让整个列表加载失败。

函数	用途
<code>json.dumps(obj, ensure_ascii=False, indent=2)</code>	转字符串； 中文要 <code>ensure_ascii=False</code>
<code>json.loads(text)</code>	解析字符串
<code>json.dump(obj, f) / json.load(f)</code>	直接读写文件对象
<code>json.dumps(obj, sort_keys=True)</code>	键排序，便于人工对比文件

`ensure_ascii=True` 是默认值，中文会变成 `\uXXXX`
写配置文件时如果忘了 `ensure_ascii=False`，文件里会是一串 `\u5f85\u529e`。程序读起来没问题，但**人打开来看不懂**——而配置文件恰恰是给人改的。

读 JSON 文件时有个固定套路：**先判文件存在、再包 try**，缺文件不该是错误：

```
1 def load_app_state() -> dict:
2     if not STATE_PATH.exists():
3         return {}
4     try:
5         return json.loads(STATE_PATH.read_text(encoding="utf-8"))
6     except (OSError, ValueError) as exc:
7         logger.warning("状态文件读取失败（%s），按空白处理", exc)
8         return {}
```

▲ 由本手册编写；项目里 `login_memory.json` 的读取就是同一形状

11.4 sqlite3：本项目唯一的持久化方案

选择 SQLite 的理由很直接：**单文件、零配置、免安装**。对一个要给别人拷过去就能跑的桌面程序来说，这是唯一合理的选项。

连接与三个必设项

```
1 def __init__(self, db_path: Path):
2     self.db_path = db_path
3     self.conn = sqlite3.connect(str(db_path), check_same_thread=False)
4     self.conn.row_factory = sqlite3.Row
5     self.conn.execute("PRAGMA foreign_keys = ON")
```

▲ `todo_db.py` · 这三行是全项目数据层的统一开头

设置	为什么必须写
<code>row_factory = sqlite3.Row</code>	不说的话查询结果是普通元组，只能按 下标 取值。设了之后可以 <code>row["name"]</code> —— 加一列不会让所有取值错位
<code>PRAGMA foreign_keys = ON</code>	SQLite 默认不检查外键 （这是历史包袱），必须每个连接显式打开
<code>check_same_thread=False</code>	允许连接跨线程使用。本项目定时器与子线程都会读库

查询：参数化、避免 SQL 注入

```

1         rows = self.conn.execute(
2             "SELECT * FROM todo_holidays WHERE day LIKE ? ORDER BY day ASC",
3             (f"{year:04d}-{month:02d}-{day:02d}",),
4         ).fetchall()
5         return [Holiday.from_row(r) for r in rows]
```

▲ todo_db.py · 问号占位符 + 参数元组

永远不要用 f-string 拼 SQL

`conn.execute(f"... WHERE name = '{name}'")` 是**注入漏洞**，而且遇到名字里带单引号（比如「O'Brien」）会直接语法错误。

正确做法就是问号占位符：**SQL 语句是固定的，数据单独作为第二个参数传**。SQLite 驱动会自己做转义。命名占位符两种写法都支持：`?` 配元组、`:name` 配字典。

批量操作：executemany

```

1     def mark_alerted(self, moments: dict[int, str]):
2         """把「这一轮已经响过」记回条目（键为条目 id，值为提醒时刻原文）。"""
3         if not moments:
4             return
5         now = _now()
6         self.conn.executemany(
7             "UPDATE todo_items SET alerted_for = ?, updated_at = ? WHERE id = ?",
8             [(text, now, item_id) for item_id, text in moments.items()],
9         )
10        self.conn.commit()
```

▲ todo_db.py · executemany + 列表推导构造参数

`executemany` 接受「一个 SQL + 一批参数」，比循环调用 `execute` 快得多（少了每轮的语句准备开销）。注意这里只 **commit 一次**，而不是每条 `commit` —— 后者会让 SQLite 做 N 次磁盘刷写。

建表：IF NOT EXISTS + executescript

```

1     def create_tables(self):
2         """建表（IF NOT EXISTS，重复调用安全）。"""
3         self.conn.executescript("""
4             /* 清单：颜色 + 图标，是「用不同颜色区分」的载体 */
5             CREATE TABLE IF NOT EXISTS todo_lists (
6                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
7                 name          TEXT    NOT NULL,
8                 color          TEXT    NOT NULL DEFAULT '#007AFF',
9                 icon           TEXT    NOT NULL DEFAULT 'list',
10                sort_order     INTEGER NOT NULL DEFAULT 0,
11                created_at      TEXT    NOT NULL,
12                updated_at      TEXT    NOT NULL
13            );
14
15            CREATE INDEX IF NOT EXISTS idx_todo_items_list
16                ON todo_items (list_id, completed, due_date);
17        """)
18        self.conn.commit()

```

▲ todo_db.py · 略作节选

IF NOT EXISTS 让建表变成幂等操作

`__init__` 里直接调 `create_tables()`，不需要先判断「数据库是新的还是老的」。这同样是幂等思想：把「首次运行」和「再次运行」写成同一条路径，就不会出现「第二次启动报错」这类只在特定顺序下出现的问题。

数据库迁移：SQLite 没有 ADD COLUMN IF NOT EXISTS

```

1     existing = {
2         row["name"]
3         for row in self.conn.execute("PRAGMA table_info(todo_items)").fetchall()
4     }
5     for column, ddl in (
6         ("alerted_for", "ALTER TABLE todo_items ADD COLUMN "
7                        "alerted_for TEXT NOT NULL DEFAULT ''"),
8         ("snooze_until", "ALTER TABLE todo_items ADD COLUMN "
9                          "snooze_until TEXT NOT NULL DEFAULT ''"),
10    ):
11         if column not in existing:
12             self.conn.execute(ddl)
13     self.conn.commit()

```

▲ todo_db.py · 加字段的标准姿势：先查后补

迁移的两条纪律

- ① **新字段必须有默认值** (`NOT NULL DEFAULT ''`) —— 否则老数据那些行就没有合法值。
- ② **加列永远只做加法**：不改名、不删列、不改类型。SQLite 的 `ALTER TABLE` 能力很弱（只支持 `ADD COLUMN` 和 `RENAME`），要改结构就得「建新表 + 拷数据 + 换名」，那是另一个量级的风险。所以设计表时多留一个空列的成本，远低于事后改结构。

排序：多级 ORDER BY

```
1 "SELECT * FROM todo_lists ORDER BY sort_order ASC, id ASC"
```

▲ todo_db.py · fetch_lists()

第二级 `id ASC` 是**稳定排序**的保证：`sort_order` 相同的两条按 `id` 排，结果不会随数据库内部顺序变化。没有它，用户会看到「顺序偶尔自己变」。这与 4.6 节里 Python 侧「用下标做第二排序键」是同一个道理。

事务与 commit

对象	作用
<code>conn.commit()</code>	提交当前事务，落盘
<code>conn.rollback()</code>	回滚未提交的改动
<code>with conn:</code>	进入时开事务，正常退出自动 <code>commit</code> 、异常自动 <code>rollback</code>
<code>conn.execute</code> 直接改数据	SQLite 驱动会 隐式开启 事务，等你 <code>commit</code>
<code>conn.close()</code>	关闭连接；未提交的改动会丢

本项目手工 commit 而不依赖 with conn:
因为待办模块的连接是跨页面共享的长连接，用 `with conn:` 会频繁开关事务，而界面操作往往是「读一下、改一下、再读一下」的连续动作，中间开事务反而容易撞上锁。
项目的做法是：**每个写方法自己以 `self.conn.commit()` 收尾**，读方法完全不碰事务。这样「一次界面操作 = 一次 `commit`」，语义清楚。

取行数据的几种方式

写法	结果	取不到时
<code>fetchall()</code>	全部行组成的列表	空列表
<code>fetchone()</code>	第一行	None
<code>fetchmany(n)</code>	最多 <code>n</code> 行	可能是短列表
<code>row["列名"]</code>	按列名取值（需设 <code>row_factory</code> ）	<code>KeyError</code>
<code>row.keys()</code>	列名列表	—
<code>conn.total_changes</code>	本连接累计改动行数	—
<code>cursor.lastrowid</code>	刚插入行的 <code>id</code>	—

```

1         cur = self.conn.execute(
2             "INSERT INTO todo_items (list_id, title, created_at, updated_at) "
3             "VALUES (?, ?, ?, ?)",
4             (list_id, title, now, now),
5         )
6         self.conn.commit()
7         return int(cur.lastrowid)

```

▲ 插入之后拿回新 id 的写法

11.5 logging：一份日志基建的最小正确实现

```

1 def setup_logging(log_dir=None, level=None, *, console=None,
2                  filename=LOG_FILENAME):
3     """配置 root logger，重复调用幂等。返回 root logger。"""
4     global _configured
5     root = logging.getLogger()
6     if _configured:
7         return root
8
9     resolved = _resolve_level(level)
10    formatter = logging.Formatter(_FORMAT, datefmt=_DATEFMT)
11    handlers = []
12
13    if log_dir is not None:
14        try:
15            log_dir = Path(log_dir)
16            log_dir.mkdir(parents=True, exist_ok=True)
17            file_handler = logging.handlers.RotatingFileHandler(
18                log_dir / filename,
19                maxBytes=MAX_BYTES,
20                backupCount=BACKUP_COUNT,
21                encoding="utf-8",
22            )
23            file_handler.setFormatter(formatter)
24            file_handler.setLevel(resolved)
25            handlers.append(file_handler)
26        except OSError as exc:
27            # 只读介质 / 权限不足等：继续走控制台，不中断启动
28            try:
29                sys.stderr.write(f"[log_setup] 日志文件不可用 ( {exc} )，仅输出到控制台\n")
30            except Exception:
31                pass

```

▲ log_setup.py · 节选

要点	做法
配 root logger	入口调一次，之后任意模块 <code>getLogger(__name__)</code> 自动生效
幂等	<code>_configured</code> 标志，重复调用直接返回，不会重复挂 handler
必须挂文件 handler	打包版 <code>console=False</code> ， <code>sys.stderr</code> 是 <code>None</code> ，只挂控制台等于日志全丢

要点	做法
轮转	<code>RotatingFileHandler(maxBytes=1MB, backupCount=3)</code> ，不会把磁盘写满
编码	<code>encoding="utf-8"</code> ，中文日志在 Windows 上才不会乱码
失败只降级	日志基建不该成为程序起不来的原因

控制台 handler 要「先探测再挂」

打包成窗口程序后 `sys.stderr` 是 **None**。此时若照旧挂 `StreamHandler(sys.stderr)`，Python 的 logging 每次 emit 都会抛 `AttributeError: 'NoneType' object has no attribute 'write'` ——然后被 logging 自己吞掉。表面平静，实际一条日志都没落。

项目的做法是写一个 `_console_stream()`：探测 `sys.stderr is None` 就返回 `None`，顺便试写一次确认可用，再去挂 handler。

```

1  logger = logging.getLogger(__name__)      # 每个模块顶部一行，不需要任何配置
2
3  # 记录异常：自动带上堆栈
4  try:
5      risky()
6  except Exception:
7      logger.exception("待办到点提醒巡检失败")
8
9  # 记录普通信息：用 %s 占位符，不要提前拼字符串
10 logger.debug("日志已初始化：级别=%s，处理器=%d 个", level, len(handlers))

```

▲ 模块内的用法

`logger.info("..." + str(x))` 与 `logger.info("...%s", x)` 的区别

后者只有真的要输出时才会做字符串格式化。日志级别是 INFO 时，一条 DEBUG 级别的格式化完全不会执行——在高频路径上这是实打实的性能差别。

项目里因此全用 `%s` 占位符形式，没有一处提前拼字符串。

`logger.exception` 只能用在自己没被处理过的异常分支里

它内部调 `sys.exc_info()`，必须处在 **except** 块中才有堆栈可打。在 **except** 块外调用它，只是记一条 ERROR 而没有堆栈——看起来一样，但排查时才发现少了最关键的信息。

11.6 re：正则的四个实用要点

```

1  _INLINE_CODE = re.compile(r"~(?:[^\s]+)"
2  _BOLD = re.compile(r"\s*~(?:[^\s]+)\s*~")
3
4  for m in PATTERN.finditer(text):
5      spans.append((m.start(), m.end(), m.group(1)))

```

▲ 本手册生成器里的用法；项目里 `markdown_view.py` 同形

要点	说明
先编译	<code>re.compile()</code> 一次，循环里复用； <code>re</code> 内部虽有缓存，显式更明确
用 <code>finditer</code> 而不是 <code>findall</code>	<code>finditer</code> 给的是 <code>Match</code> 对象，能拿到 <code>start()</code> / <code>end()</code> / 分组
用 <code>re.escape()</code> 转义字面量	搜用户输入或文件名时必做，否则 <code>*</code> 会被当语法
原始字符串	正则一律写 <code>r"..."</code> ，否则反斜杠要先过 Python 一层

多分组正则 + `findall` 会得到元组列表

`re.findall(r"(\r\n|\n)", text)` 返回的是**元组列表**而不是字符串列表，于是 `hits[0]` 拿到的是 `(' \n',)`。本项目在写字节补丁脚本时踩过一次。

要精确控制就一律用 `finditer`：`for m in re.finditer(pat, text)`，然后 `m.group(0)` 取整体、`m.group(1)` 取第一个分组，还能用 `m.start()` / `m.end()` 定位。

另一个与字节打交道时的常见错误：**在字节串上做正则时，模式也要是字节串**。项目里查文件里有没有三引号的脚本就是把模式和 `re.escape()` 都编码成 `bytes`：

```
1 for m in re.finditer(re.escape(b'"\\"'), raw):
2     ...
```

▲ 在 `bytes` 上做正则，模式和输入都必须是 `bytes`

11.7 其它常用模块

模块	用途	项目里的例子
<code>os</code>	环境变量、路径	<code>os.environ.get(ENV_LEVEL, "")</code> 覆盖日志级别
<code>sys</code>	解释器相关	<code>sys.frozen</code> 判打包、 <code>sys.path</code> 加搜索路径
<code>subprocess</code>	启动外部程序	工具包启动 <code>exe</code> 、 <code>explorer</code> 打开目录
<code>shutil</code>	文件复制删除	导入图片时复制到数据目录
<code>webbrowser</code>	打开浏览器	点链接打开文档
<code>zipfile</code>	压缩包	读 Excel (xlsx 本质是 zip)
<code>hashlib</code>	摘要	密码哈希 <code>hashlib.sha256(...).hexdigest()</code>
<code>calendar</code>	日历计算	<code>calendar.monthrange()</code> 取月末
<code>time</code>	计时	<code>time.perf_counter()</code> 测耗时
<code>ctypes</code>	调 Windows API	单实例互斥、 <code>PrintWindow</code> 截图

```
1 def hash_password(password: str) -> str:
2     return hashlib.sha256(password.encode("utf-8")).hexdigest()
```

▲ `main.py` · 密码哈希（自用工具的强度取舍，见下方说明）

SHA-256 直接哈希密码，在生产环境是不够的

没有加盐、没有慢哈希，字典攻击很容易。本项目是单机自用工具、只防「别人翻开数据库文件能看到明文」，这个强度按需求是够的。

真要正经存密码，用 `hashlib.scrypt` 或 `bcrypt`，它们把每一条密码的哈希成本抬到几十毫秒，暴力破解就不可行了。这一段的价值不在代码，而在「知道自己选的是什么强度」。

```
1 result = subprocess.run(  
2     [str(exe_path), "--version"],  
3     capture_output=True, text=True, timeout=15,  
4     creationflags=subprocess.CREATE_NO_WINDOW,    # 不弹黑框  
5 )  
6 if result.returncode == 0:  
7     version = result.stdout.strip()
```

▲ 运行外部命令并取输出（参数用列表形式，不要拼成一个字符串）

`subprocess` 的三条纪律

- ① 参数用列表：`["powershell.exe", "-Command", cmd]`。用 `shell=True` 加拼字符串等于把命令行注入的口子开着。
- ② 一定要 `timeout`：外部程序卡住会连带界面卡死。
- ③ Windows 上给 `CREATE_NO_WINDOW`：否则每跑一次都会闪一个黑框，对一个 GUI 程序来说非常难看。

第十二章 GUI 相关的 Python 语法

这一章不讲界面设计（那是另一个话题），只讲用 Python 写 Tkinter 时会用到的语言特性：回调、闭包、定时器、事件、变量类，以及控件生命周期带来的那几个专属异常。

12.1 界面为什么用类组织

```
1 class TodoPage(ttk.Frame):
2     def __init__(self, master, db=None, *, on_status=None):
3         super().__init__(master)
4         self.db = db
5         self.current_list_id = 0
6         self._list_rows = {}
7
8         self._build()
9         self.refresh_all()
```

▲ todo_page.py · 页面类的构造骨架

用类的理由有两条，都很实际：

1. **状态要有地方放。** `self.current_list_id`（当前选中的清单）、`self._list_rows`（行控件索引）必须活到下一次交互——装在函数局部变量里，函数一返回就没了。
2. **回调要能找回来。** 按钮的 `command=self.on_click` 是一个绑定方法，它自带 `self`，所以点下去时能拿到这套状态。用纯函数就得靠全局变量。

为什么页面继承 `ttk.Frame` 而不是 `object`

`ttk.Frame` 本身就是一个容器控件。继承它之后，`TodoPage` 实例可以直接 `pack()` / `grid()` 到父容器里，就和一个普通 `Frame` 一样用：`self.todo_view = TodoPage(container, db); self.todo_view.pack(fill="both")`。如果继承 `object`，就得自己再持有一个 `Frame` 并把它暴露出去，多一层转发。

注意构造签名里的 `db=None`：数据层是「注入」进来的，不是页面自己 `new` 的。这是 7.8 节「组合」的直接收益——测试时塞一个替身进去就行，不需要真数据库。

12.2 回调：传函数对象，不传调用结果

```
1 dialog = TodoAlertDialog(
2     self, items, db=self.todo_db,
3     on_changed=self._after_todo_alert,    # 传函数对象，不加括号
4     on_open=self._open_todo_item,
5 )
```

▲ main.py · 三个回调全是具名方法名

正确	错误	错误的结果
<code>command=self.save</code>	<code>command=self.save()</code>	绑定时就执行一次，回调变成 <code>None</code>
<code>bind("<Enter>", self._on_enter)</code>	<code>bind("<Enter>", self._on_enter())</code>	同上
<code>command=lambda: self.save(1)</code>	<code>command=lambda x: self.save(1)</code>	参数对不上，点击时报错
<code>bind("<Enter>", lambda e: f(e))</code>	<code>bind("<Enter>", lambda: f(e))</code>	Tkinter 会传事件参数，参数个数不符

bind 的回调一定会收到一个事件对象

`widget.bind("<Enter>", handler)` 里的 `handler` 会被 Tkinter 调用成 `handler(event)`。所以回调签名必须能收下 一个参数 —— 这就是为什么项目里到处是 `lambda e: self.set_hover(True)` 里的那个 `e`。如果用的是具名方法（`self._invoke`），它的签名就得是 `def _invoke(self, event=None)`。写成 `def _invoke(self)` 也能跑 `command=`，但接 `bind` 就会 `TypeError` —— 所以项目里统一给事件处理方法加 `event=None`。

```
1 self.bind("<Enter>", lambda e: self.set_hover(True))
2 self.bind("<Leave>", lambda e: self.set_hover(False))
3 if command is not None:
4     self.bind("<Button-1>", self._invoke)
```

▲ `ui_components.py` · 两种回调用法并存

12.3 after: Tkinter 的定时器

```
1 TODO_TICK_MS = 30 * 1000          # 30 秒
2
3 def todo_alert_check(self):
4     """巡检一次「有没有提醒该响了」。
5
6     自己续期，所以只要程序在跑就一直有效 —— 窗口缩到托盘也一样
7     （mainloop 还活着），人不在电脑前也不会漏掉。
8     """
9     if getattr(self, "_exiting", False):
10         return
11     try:
12         self._run_todo_alerts()
13     except Exception:
14         logger.exception("待办到点提醒巡检失败")
15     finally:
16         self.after(TODO_TICK_MS, self.todo_alert_check)
```

▲ `main.py` · 自续期定时器



▲ 就是上面那个 30 秒定时器弹出来的东西：屏幕右下角的到点提醒窗。`-topmost` 压过别的窗口，但不 `grab_set`，所以不会把正在打字的人从输入框里拽出来。

方法	作用
<code>widget.after(ms, func)</code>	延迟 ms 毫秒调用一次
<code>widget.after(ms, func, *args)</code>	带参数调用
<code>widget.after_idle(func)</code>	界面空闲时调用
<code>handle = widget.after(...)</code>	返回句柄，可以取消
<code>widget.after_cancel(handle)</code>	取消（退出前必须做，否则回调会打到已销毁的控件上）
<code>widget.after_cancel("all")</code>	取消该控件上全部定时任务
<code>widget.update()</code> / <code>update_idletasks()</code>	强制立刻处理一次事件队列（截图、测试时用）

自续期定时器的三条纪律

- ① 续期必须放 `finally` —— 否则一次异常就让定时器永久死掉（见 8.1）。
- ② 退出前要停 —— 项目里用 `self._exiting` 标志：退出流程一开始就置位，下一轮巡检看到标志直接 `return`，不会再去碰正在销毁的控件。
- ③ 回调出错要有人接住 —— Tkinter 的事件循环里抛出的异常不会让程序退出，只会打到 `stderr`（打包版等于吞掉）。所以回调里必须自己 `try/except` + 记日志。

为什么不用 `threading.Timer` 或独立线程

Tkinter 不是线程安全的：只有创建控件的那一个线程能碰控件。从子线程里调 `widget.config(...)` 可能看起来正常、也可能在几分钟后随机崩掉 —— 这类 bug 极难复现。
`after()` 跑在主线程的事件循环里，天然没有这个问题。要在界面程序里做定时任务，就用 `after`。

12.4 事件绑定与事件对象

事件	触发
<Button-1> / <ButtonRelease-1>	鼠标左键按下 / 松开
<B1-Motion>	按住左键拖动
<Double-Button-1>	双击
<Enter> / <Leave>	鼠标进入 / 离开控件
<KeyRelease>	按下一个键
<Return>	回车
<FocusOut>	失去焦点（表单自动保存常绑它）
<Configure>	控件尺寸变化
<<ComboboxSelected>>	虚拟事件（ttk 特有），注意是双尖括号

```
1 class _DragEvent:
2     """拖动回调用得到的字段就这两个；自己造一个替身，测试时就不需要真鼠标。"""
3
4     def __init__(self, y_root: int):
5         self.y_root = y_root
6
7
8 page._list_drag_press(ids[0], _DragEvent(base + mid(0)))
9 page._list_drag_motion(ids[0], _DragEvent(base + mid(2)))
10 page._list_drag_release(ids[0], _DragEvent(base + mid(2)))
```

▲ scripts/test_todo_ui.py · 用替身对象模拟事件

这段是测试写法上的一个要点：拖动处理只用到事件对象上的 `y_root` 一个字段，那么测试就不必真的造鼠标事件，传一个只有该字段的小对象即可。这叫「鸭子类型」的实用面——不要求参数是某个类，只要求它有那个属性。结果是拖动的回归测试完全不需要真实鼠标操作，跑起来稳定又快。

事件对象上常用的字段

`event.x / event.y` 是相对控件左上角的坐标；`event.x_root / event.y_root` 是相对屏幕的坐标；`event.widget` 是触发事件的控件；`event.keysym` 是按键名（如 `"Return"`）；`event.width / event.height` 在 `<Configure>` 里给出新尺寸。

拖动计算要用 `y_root`（屏幕坐标），因为它不受「被拖动的行自己在移动」影响；用 `event.y` 会在行移动之后跟着漂。

12.5 变量类：StringVar / IntVar / BooleanVar

```
1 self.keyword_var = tk.StringVar()
2 entry = ttk.Entry(bar, textvariable=self.keyword_var)
3 entry.bind("<KeyRelease>", lambda e: self._on_search())
4 ...
5 def _on_search(self):
6     keyword = self.keyword_var.get().strip()
```

▲ todo_page.py · 搜索框的两种用法并存

写法	取值	适用
<code>textvariable=var</code>	<code>var.get()</code> / <code>var.set(x)</code>	需要程序主动改控件内容时
直接 <code>entry.get()</code>	从控件读	只读用户输入时更简单
字符串	<code>tk.StringVar()</code>	文本
整数	<code>tk.IntVar()</code>	复选框、计数器
布尔	<code>tk.BooleanVar()</code>	<code>Checkbutton</code>
双击	<code>tk.DoubleVar()</code>	滑块数值

变量类是「双向绑定」的实现方式

`textvariable=` 之后，改 `var` 会立刻改控件，改控件也会立刻改 `var`。本项目只在「需要程序回写控件」的地方用它（比如搜索框要能被清空）。纯读取输入的场景直接 `entry.get()` 更省事 —— 少一个需要维护的对象。一个易错点：`IntVar` 在输入框为空或非数字时 `get()` 会抛 `TclError`，因为 Tk 存的是字符串、转 `int` 失败。所以读数字一律要包一层兜底。

12.6 控件生命周期与 `TclError`

```
1 key = tuple(base)
2 cached = _STRIKE_FONTS.get(key)
3 if cached is not None:
4     try:
5         cached.actual()           # 探一次活：解释器换过之后旧对象就是废的
6         return cached
7     except tk.TclError:
8         _STRIKE_FONTS.pop(key, None)
```

▲ todo_page.py · `strike_font()` 的缓存探活

控件被销毁之后，任何 `cget / config` 都会抛 `TclError`

报错长得像 `TclError: invalid command name ".!todopage.!frame.!label3"`。触发场景很多：定时器回调打在已关闭的窗口上、在 Tk 解释器销毁后复用缓存的 `Font` 对象、跨测试用例复用了上一个 `Tk()` 根窗口的控件。

应对方式有两种，项目里都用了：

① 先判存在：`if widget.winfo_exists(): widget.destroy()`；

② 探活后再用：像上面这样先调一个无害方法，抛 `TclError` 就说明对象已废，清掉缓存重建。

永远不要缓存 Tk 对象而不做存活检查 —— 本项目的字体缓存就是这么修的。

```
1 def _close_todo_alert(self):
2     """关掉还挂着的提醒窗（开会错过的那一批不应一直压在最上层）。"""
3     dialog = getattr(self, "_todo_alert_win", None)
4     self._todo_alert_win = None          # 先置空：让本方法可重复调用
5     if dialog is None:
6         return
7     try:
8         if dialog.winfo_exists():
9             dialog.destroy()
10    except Exception:
11        pass
```

▲ main.py · 关闭窗口的标准姿势：先置空引用、再判存在、最后包 try

winfo 系列：询问控件的实际状态

方法	返回
<code>winfo_exists()</code>	控件是否还活着
<code>winfo_width()</code> / <code>winfo_height()</code>	当前实际尺寸（布局算完之后才是真实值）
<code>winfo_reqwidth()</code> / <code>winfo_reqheight()</code>	控件自己请求的尺寸
<code>winfo_x()</code> / <code>winfo_y()</code>	相对父容器的左上角坐标
<code>winfo_rootx()</code> / <code>winfo_rooty()</code>	相对屏幕的坐标
<code>winfo_children()</code>	子控件列表
<code>winfo_screenwidth()</code> / <code>winfo_screenheight()</code>	屏幕尺寸
<code>winfo_ismapped()</code>	是否真的被显示出来了

`winfo_width()` 在布局计算前返回的是 1（不是错，是还没算）

刚 `pack()` 完立刻问尺寸，拿到的往往是 1。必须等 Tk 处理过一次事件队列，也就是调用 `update_idletasks()` 或 `update()` 之后才有真实值。

测试里所有「量尺寸」的断言前面都必须先 `update()`，否则断言的是「还没布局」而不是「布局结果」。

另外 `winfo_ismapped()` 有个特别之处：主窗口 `withdraw()` 时，所有控件都返回 0。所以拿它判断「控件有没有被饿死」之前，必须先 `deiconify()`。

12.7 一个跨语言的坑：Tk 会「饿死」控件

```
1         self.vsb = ttk.Scrollbar(self, orient="vertical")
2         self.vsb.pack(side="right", fill="y")           # 先 pack 滚动条
3
4         self.canvas = tk.Canvas(self, bg=bg, highlightthickness=0, bd=0,
5                                   yscrollcommand=self.vsb.set)
6         self.canvas.pack(side="left", fill="both", expand=True) # 后 pack 画布
```

▲ todo_page.py · ScrollArea 里 pack 的顺序

这个坑不属于 Python 语法，但每次都让人以为是语法问题

`pack` 是按调用顺序分配剩余空间的。如果先 `pack` 一个 `fill="both"`，`expand=True` 的控件，它会把空间全部吃掉，后面 `pack` 的固定尺寸控件只分到 **0 像素**，于是 Tk **根本不映射它**——代码明明写了，界面上就是不存在。这不是「被裁掉一点」，而是**完全不存在**。判定口径是：`winfo_manager()` 非空 且 `winfo_ismapped() == 0`。
规矩：先 pack 固定尺寸的控件，最后 pack 那个 `expand=True` 的内容区；滚动条要先于它管理的滚动区域。

本章小结：GUI 里最该记住的四条

- ① 回调传函数对象 (`self.save`)，不传调用结果 (`self.save()`)；
- ② 定时任务用 `after` + `finally` 续期 + 退出标志，不要用线程；
- ③ 任何缓存的 Tk 对象在用之前都要探活，控件销毁后一律 `TclError`；
- ④ 别在模块顶层建窗口或控件——那会让 `import` 有副作用，测试也没法隔离。

第十三章 工程约定与陷阱

这一章是那些不属于任何单一语法点、但每个真实项目都会撞上的东西。它们大多与「Python 在 Windows 上 + 中文 + 打包发布」这个组合有关。

13.1 行尾：CRLF 与 LF

场景	行尾	说明
Windows 上的多数编辑器	CRLF (<code>\r\n</code>)	本项目应用源码就是这种
Git Bash / 跨平台工具	LF (<code>\n</code>)	本项目 <code>scripts/*.py</code> 与 <code>*.md</code> 是这种
Python 文本模式读取	统一翻译成 <code>\n</code>	<code>universal newlines</code> ，看不见 CRLF
Python 文本模式写入	<code>\n</code> 被翻译成平台默认	Windows 上就是 CRLF —— 这是 <code>write_text</code> 的陷阱
Python 二进制模式	原样进出	字节级补丁必须走这条

关键点是第二行与第四行：Python 的文本模式会让行尾「看起来统一」，所以你在代码里从没感觉到 CRLF 的存在；但写回磁盘时它又会按平台转换，于是「读出来再写回去」这个动作本身就会改文件。

`Path.write_text` 在 Windows 上会把整个文件变成 CRLF

本项目实际踩过：一个纯 LF 的测试脚本用 `write_text` 写回之后，946 行全部变成 CRLF，`git diff` 显示「946 增 / 778 删」，真实的逻辑改动被淹没在里面。

要保住原行尾只有一条路：`read_bytes()` + `write_bytes()`，中间需要处理文本时用 `decode("utf-8")`，处理完再 `encode("utf-8")`，全程不让行尾经过文本模式的翻译。

```
1 raw = p.read_bytes() # 字节读：CRLF 保持原样
2 assert raw.count(b"\r\n") == 0, "这个文件应该是纯 LF" # 先确认现状
3 before = raw.count(b"\n")
4 text = raw.decode("utf-8") # 现在是字符串，里面的换行是 "\n"
5 text = text.replace(old, new, 1) # 只改目标片段
6 out = text.encode("utf-8") # 字节写回：不产生任何 CRLF
7 assert out.count(b"\n") == before + added # 行数变化符合预期
8 p.write_bytes(out)
```

▲ 纯 LF 文件的补丁骨架

完整规则（本项目所有改写源码的操作都遵守）

- 1. 先查明行尾现状：raw.count(b"\r\n") 与 raw.count(b"\n") 相等 → 纯 CRLF；后者减前者得到的就是「单独 LF」的数量。
- 2. 纯 CRLF 文件：text.replace("\n", "\r\n") 写回，并断言 out.count(b"\r\n") == out.count(b"\n")。
- 3. 纯 LF 文件：绝对不要经过 write_text，只走 write_bytes。
- 4. 混排文件（如 main.py）：先记下「单独 LF」的数量（167 个），补丁锚点带上 \r\n，写完复核这个数量没变。
- 5. 收尾一律用 git diff --numstat 复核变更行数 —— 数字不对说明行尾被动过。

13.2 文本 vs 字节：什么时候必须用字节

任务	该用的 API	理由
读写配置 / 日志	read_text / write_text	人看的文本，行尾无所谓
改写源码文件	read_bytes / write_bytes	必须保住原行尾
解析二进制格式	read_bytes	图片、Excel、zip
网络 / 子进程 IO	bytes	协议是字节流
哈希、加解密	bytes	算法定义在字节上
写 CSV / 文本导出	open(..., newline="")	让 csv 模块自己管行尾

csv 模块必须配 newline=""

csv.writer 自己会写 \r\n，而文本模式又会把 \n 再翻一遍，结果是每行多一个空行。正确写法是 open(path, "w", encoding="utf-8-sig", newline="")。这是 Python 官方文档特别标注的一条。

encoding="utf-8-sig" 是什么时候该用

utf-8-sig 会在文件开头写 BOM (EF BB BF)。Excel 打开 CSV 时

靠这个 BOM 判断编码，没有它中文会乱码。所以：给用户用 Excel 打开的 CSV 用 utf-8-sig；程序自己读写的 JSON / 日志 / 源码统统用普通 utf-8（加 BOM 反而会让严格的解析器报错）。

本项目 tools_page.py 里的 CSV 导出就是这么分的。

13.3 编码：统一 UTF-8，一处都不要例外

```
1 # 源码文件
2 # -*- coding: utf-8 -*-
3
4 # 读写文件
5 path.read_text(encoding="utf-8")
6 path.write_bytes(text.encode("utf-8"))
7
8 # 日志
9 RotatingFileHandler(..., encoding="utf-8")
10
11 # 子进程
12 subprocess.run(..., encoding="utf-8", errors="replace")
13
14 # 控制台（打包版 stderr 可能没有）
15 stream.reconfigure(errors="replace")
```

▲ 项目里的编码设置集中在这几处

坑	表现	应对
Windows 默认编码是 cp936	读写文件不指定 <code>encoding</code> 时中文可能乱码或报错	每个 open 都写 <code>encoding="utf-8"</code>
控制台打印中文	<code>UnicodeEncodeError</code> 或显示问号	<code>errors="replace"</code> 放宽
BOM	JSON 解析失败、字符串开头多一个不可见字符	程序内部一律普通 utf-8
源码里有非法字节	<code>SyntaxError: Non-UTF-8 code</code>	确保编辑器保存为 UTF-8 无 BOM

13.4 时间：本项目明确选了「本机时间」

做法	含义	适用
<code>datetime.now()</code>	本机当前时间, naive	单机自用工具（本项目）
<code>datetime.utcnow()</code>	UTC 时间, naive	已不推荐，用下面那个
<code>datetime.now(timezone.utc)</code>	带时区的 UTC	需要跨机器一致时
<code>datetime.now().timestamp()</code>	Unix 时间戳（秒）	存库、跨系统比较
<code>date.today()</code>	本机日期	只关心「哪一天」时

naive 时间的边界在哪

naive datetime 的含义是「这台机器认为的本地时间」。它有三种情况会出问题：

- ① 换机器：同一份数据在另一个时区的电脑上打开，时间含义就变了；
- ② 夏令时：本地时间在切换日会出现重复或缺失的一小时；
- ③ 与外部系统对时间：服务器给的是 UTC，直接比较会差几小时。

本项目的场景是「单机、自用、只在本地比较」，所以 naive 时间成立。但这个结论要写下来（就像这条注释一样），否则将来接了云同步功能，没人知道当初是「有意选的」还是「没想到」。

13.5 数值：先把误差摆到台面上

```
1 print(0.1 + 0.2)           # 0.30000000000000004
2 print(0.1 + 0.2 == 0.3)    # False
3 print(round(2.675, 2))     # 2.67 —— 不是四舍五入的错，是 2.675 本身就存不准
4 print(1 / 3)               # 0.3333333333333333
```

▲ 这些结果都是「正确」的，浮点数就是这样

需求	做法
显示两位小数	<code>f"{v:.2f}"</code>
比较两个浮点是否相等	<code>abs(a - b) < 1e-9</code>
金额计算	用 <code>int</code> 存分，或 <code>decimal.Decimal("0.1")</code>
只要整数结果	<code>//</code> （向下取整）、 <code>int()</code> （向零取整）、 <code>round()</code> （银行家舍入）
统计求和顺序敏感	<code>math.fsum()</code> 精度更好

`round()` 是「银行家舍入」，不是四舍五入
`round(0.5)` 得 0、`round(1.5)` 得 2 —— 它舍入到最近的偶数。这是 IEEE-754 的默认策略，用来避免大量数据相加时的系统性偏差。要真正的四舍五入，用 `decimal.Decimal(...).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)`。

13.6 静态检查：pyflakes 与它的口径

工具	查什么	本项目
pyflakes	未使用的导入 / 变量、未定义的名字、f-string 问题	在用，作为回归基线
ruff	pyflakes + pycodestyle + 很多规则，极快	待办清单上
mypy / pyright	类型检查	待办清单上
black	格式化	未用（现有风格已统一）

pyflakes 的口径要限定在「有效范围」
项目根目录有一批 `_` 开头的临时脚本，噪声很大，把它们算进来会让告警数虚高。本项目的有效基线只统计「非 `_` 开头 + `embedded_admin_tools/`」，当前是 7 条旧账，全是「赋值未使用」这类无关痛痒的。
关键不是「零告警」，而是「告警数不再增加」—— 回归时对比数字，新增一条就要查。

本项目补的两个专用检查脚本

`pyflakes` 有两个盲区，项目为此写了单独脚本：

`check_name_scope.py` —— 查 `except` 块里的名字在块外被引用（Python 3 会在 `except` 结束时删除 `except E as e` 里的 `e`）以及闭包作用域问题；

`project_audit.py` —— 用 AST 统计函数长度、嵌套深度、重复代码。

`pyflakes` 不是全覆盖，知道它漏什么，比用到它更重要。

13.7 打包：Python 特有的三个认知

```
1 ExpiryManager_fixed.spec
2 |—— Analysis(['main.py'])      分析依赖图，收集所有 import
3 |—— PYZ                        把 .pyc 压进一个归档
4 |—— EXE                        bootloader + PYZ + 数据文件
5 |—— (onfile 模式没有 COLLECT 目录，全部塞进单个 exe)
```

▲ PyInstaller onfile 的产物结构

认知	后果
漏了模块不报错	只在运行版炸。所以要「反向核对」：打开 exe 归档验证符号在不在
<code>hiddenimports</code> 要手工补	动态导入（ <code>importlib</code> 、字符串名字）分析不到，必须列出
路径会变	<code>__file__</code> 指向临时解包目录，数据目录要用 <code>sys.executable</code> 定位
没有控制台	<code>console=False</code> 时 <code>sys.stderr</code> 是 <code>None</code> ，日志必须落文件
首次启动慢	onfile 要先解包到临时目录，体积越大越慢

「反向核对」怎么做

打开 exe 的归档，把源码编译后的字节码取出来，检查关键函数名在不在：

`CArchiveReader(exe) → open_embedded_archive("PYZ.pyz") → extract("todo_db")`，然后递归遍历 `code object` 的 `co_names` / `co_consts`（`repr(code)` 看不到函数名）。入口脚本 `main` 不在 `PYZ` 里，得从 `CArchive` 的 `main` 条目取。

本项目把它固化成 `scripts/check_exe.py`，每次打包后跑一遍。

打包脚本最大的危险是「删掉真实数据」

很多打包脚本第一步是 `rmdir /s /q dist`，之后只把少数几个文件拷回去 —— 于是用户积累的运行期数据永久丢失（数据库、上传的图片、登录记忆）。本项目因此不用那个脚本，改用「只备份数据库与工具目录到临时位置、只删 exe」的安全版本。

任何「清理后重建」的脚本，都要先问一句：重建得回来吗？

附录 A · 语法速查卡

这份附录是一页纸级别的浓缩。适合打印出来放在手边，或者在你「明明记得有这么一个语法但想不起来怎么写」的时候翻。

A.1 数据类型与字面量

	字面量	可变	可哈希	常用取值
int	42、0x1F、0b1010、1_000_000	否	是	整数
float	3.14、1e-9	否	是	小数
bool	True / False	否	是	int 的子类
str	'a'、"a"、f"{x}"、r"\d"	否	是	文本
bytes	b"abc"	否	是	字节日志、二进制
list	[1, 2]	是	否	有序收集
tuple	(1, 2)、1, 2	否	是	固定组合、字典键
dict	{'a': 1}	是	否	键值映射（保插入序）
set	{1, 2}、set()	是	否	去重、存在性
None	None	否	是	「没有值」
frozenset	frozenset({1})	否	是	不可变集合

A.2 字符串与容器操作

类别	操作
字符串	s.strip() s.split(sep) s.replace(a,b) s.startswith(p) s.endswith(p) s.casefold() s.zfill(2) s.join(seq) s in t s[1:5] s[::-1]
列表	append extend insert remove pop clear index count sort reverse copy
字典	d.get(k, dflt) d.setdefault(k, v) d.update(o) d.pop(k, dflt) d.keys() d.values() d.items() d1 d2
集合	add discard remove union intersection difference issubset
通用	len(x) x in c max/min(c) sum(c) sorted(c) reversed(c) enumerate(c) zip(a, b) any(c) all(c) list(c) tuple(c) set(c) dict(pairs)

A.3 控制流与函数

	写法
条件	<code>if / elif / else</code> ; 三目 <code>a if c else b</code>
循环	<code>for x in c:</code> / <code>while c:</code> ; <code>break / continue / pass</code> ; <code>for ... else:</code>
异常	<code>try / except E / else / finally</code> ; <code>raise E(...)</code> / <code>raise ... from e</code>
函数定义	<code>def f(a, b=1, *args, c, **kw) -> T:</code>
仅关键字	<code>def f(a, *, b):</code> —— <code>b</code> 只能用 <code>b=</code> 传
返回多值	<code>return a, b</code> (实为元组)
lambda	<code>lambda x: x + 1</code> —— 只能是一个表达式
装饰器	<code>@deco</code> 写在 <code>def</code> 上方; <code>functools.wraps</code> 保住元信息
闭包	内层函数读外层变量; 写要 <code>nonlocal / global</code> ; 绑定用 <code>lambda i=i:</code>

A.4 类

	写法
定义	<code>class C(Base):</code>
构造	<code>def __init__(self, ...):</code> 第一句是 <code>super().__init__()</code>
实例 / 类属性	<code>self.x = 1</code> / 类体里 <code>x = 1</code>
计算属性	<code>@property def x(self):</code> 用 <code>obj.x</code> 读
备用构造	<code>@classmethod def from_row(cls, row):</code>
无状态工具	<code>@staticmethod def helper():</code>
数据类	<code>@dataclass</code> ; 可变默认值用 <code>field(default_factory=list)</code>
魔术方法	<code>__init__ __repr__ __eq__ __len__ __iter__ __enter__ __exit__</code>
继承检查	<code>isinstance(x, C) / isinstance(D, C) / C.__mro__</code>

A.5 模块与导入

	写法
导入模块	<code>import m</code> $\rightarrow$ <code>m.f()</code>
导入名字	<code>from m import f, g</code> $\rightarrow$ <code>f()</code>
别名	<code>import tkinter as tk</code> / <code>from x import y as z</code>
相对导入	<code>from . import x</code> / <code>from .sub import y</code>
脚本守卫	<code>if __name__ == "__main__":</code>

	写法
延迟导入	把 <code>import</code> 放进函数体（省启动时间）
加搜索路径	<code>sys.path.insert(0, str(Path(__file__).resolve().parent.parent))</code>
打包路径	<code>Path(sys.executable).parent if getattr(sys, "frozen", False) else Path(__file__).parent</code>

A.6 常用标准库

需求	写法
路径	<code>Path(a) / "b" / .parent.name.stem.suffix.resolve().exists()</code>
建目录	<code>p.mkdir(parents=True, exist_ok=True)</code>
读文本	<code>p.read_text(encoding="utf-8")</code>
保行尾写回	<code>p.write_bytes(text.encode("utf-8"))</code>
当前时间	<code>datetime.now() / date.today()</code>
时间格式化	<code>dt.strftime("%Y-%m-%d %H:%M")</code> → "2026-09-21 14:05"
日期运算	<code>d + timedelta(days=1)</code> ；月末天数 <code>calendar.monthrange(y, m)[1]</code>
JSON	<code>json.dumps(o, ensure_ascii=False) / json.loads(s)</code>
SQLite	<code>sqlite3.connect(p) + conn.row_factory = sqlite3.Row</code>
日志	<code>logging.getLogger(__name__)</code> → <code>logger.exception("...")</code>
子进程	<code>subprocess.run(..., capture_output=True, text=True, timeout=15)</code>
环境变量	<code>os.environ.get("NAME", "")</code>
正则	<code>re.compile(r"...")</code> → <code>.finditer(text) / .search / .sub</code>
哈希	<code>hashlib.sha256(b).hexdigest()</code>

A.7 高频惯用法

目的	写法
计频	<code>d[k] = d.get(k, 0) + 1</code>
设默认不覆盖	<code>d.setdefault(k, v)</code>
兜底默认（小心吞掉 0）	<code>x = value or DEFAULT</code>
提取一列	<code>[row["name"] for row in rows]</code>
过滤	<code>[x for x in xs if 条件]</code>
建索引	<code>{x.id: x for x in xs}</code>

目的	写法
去重保序	<code>list(dict.fromkeys(xs))</code>
多条件排序	<code>sorted(xs, key=lambda x: (-x.score, x.name))</code>
稳定二级排序	<code>sorted(xs, key=lambda x: (x.a, x.id))</code>
并列遍历	<code>for a, b in zip(xs, ys):</code>
带下标遍历	<code>for i, x in enumerate(xs, 1):</code>
交换	<code>a, b = b, a</code>
安全取值	<code>d.get(k, "")</code> 而不是 <code>d[k]</code>
幂等追加	追加前先 <code>find</code> 标记, 命中就跳过
判 None	<code>if x is None:</code> (不要用 <code>== None</code>)
空值判真	<code>if not x:</code> 会连 <code>0 / "" / []</code> 一起匹配

附录 B · 模块清单

下表是生成这份手册时对源码目录**现算**的结果，顺序按文件行数从大到小。职责取自各模块自己的 docstring 首句。

模块	行数	类	函数	职责
main.py	5444	9	39	ExpiryManager - 个人系统（到期管理器）v2.0
tools_page.py	4485	6	13	工具包页面（重写版：分类+包管理、相对路径、图标抽取）
todo_page.py	2165	3	3	待办事项模块 - 界面层（View Layer）
study_notes_window.py	1844	3	3	学习笔记主窗口
embedded_admin_tools/api_demo_window.py	1607	1	0	（无模块 docstring）
scripts/test_tools_launcher.py	1576	0	36	系统工具箱「启动器」回归测试。
todo_db.py	1545	5	10	待办事项模块 - 数据层（Data Layer）
embedded_admin_tools/market_quote_window.py	1458	1	0	（无模块 docstring）
scripts/test_todo_ui.py	1355	1	29	待办页面 UI 回归测试：真实窗口 + 样例数据 + 逐项断言。
adb_page.py	1080	1	0	ADB 工具箱
scripts/test_todo.py	976	0	17	待办 / 提醒事项模块回归测试（数据层 + 节假日引擎，纯逻辑、不碰 Tk）。
embedded_admin_tools/error_code_service.py	929	1	2	错误码服务 - returnValue.json 内嵌版本
study_demo_window.py	925	2	0	Python 学习辅助模块
study_notes_db.py	895	4	2	学习笔记模块 - 数据层（Data Layer）
scripts/manual_engine.py	838	3	14	Python 语法手册的排版引擎 —— 把「块列表」渲染成一份带目录、页码、书签的 PDF。
qa_work_log_page.py	774	5	0	Q&A 问答 + 每周工作纪要 + 练习记录 页面
tools_db.py	768	1	38	工具库数据库
qa_work_log_db.py	707	8	4	Q&A 问答 + 每周工作纪要 + 练习记录 数据层
qa_extras.py	668	4	4	Q&A + 工作纪要 + 练习记录 扩展功能
ui_theme.py	592	3	0	全局 UI 主题定义。

模块	行数	类	函数	职责
<code>embedded_admin_tools/crypto_window.py</code>	573	1	0	(无模块 docstring)
<code>account_windows.py</code>	542	2	1	共享账户管理相关窗口。
<code>embedded_admin_tools/interface_service.py</code>	527	1	2	接口服务 - <code>interfaces.json</code> 内嵌版本
<code>markdown_view.py</code>	509	0	5	Markdown 渲染 (Typora 风格)
<code>expiry_dialogs.py</code>	492	3	2	到期管理相关弹窗。
<code>ui_components.py</code>	482	2	10	统一 UI 组件层。
<code>embedded_admin_tools/login_checker_window.py</code>	464	1	0	(无模块 docstring)
<code>baidu_disk_window.py</code>	463	2	0	百度网盘资料库管理窗口
<code>tools_launcher.py</code>	408	0	23	<code>tools_launcher.py</code> —— 工具启动器的纯逻辑层。
<code>embedded_admin_tools/app_icon_service.py</code>	398	0	1	主程序图标内嵌服务
<code>console_page.py</code>	397	1	0	<code>console_page.py</code> - 控制台页面(日志输出 + 命令执行 + 工具维护)
<code>todo_icons.py</code>	375	0	47	待办模块的自绘图形资产。
<code>app_icons.py</code>	373	1	23	应用图形资产: 应用标记 + 导航线性图标。
<code>scripts/project_audit.py</code>	367	1	10	项目静态审计脚本 — 只读分析, 不修改任何被分析文件。
<code>scripts/prune_unused_imports.py</code>	343	0	10	未使用导入裁剪器 —— 依据 <code>pyflakes</code> 结果精确删除导入名, 只读预览为默认。
<code>nav_sidebar.py</code>	336	1	0	左侧分组导航侧栏 (自绘)。
<code>scripts/test_notes_markdown.py</code>	323	0	6	笔记页排版回归测试: Markdown 预览 tag + 编辑器外框 + 底部预览面板。
<code>scripts/test_notes_editor.py</code>	304	1	3	笔记编辑器回归测试: 格式工具栏动作 + 源码淡化 + 语法速查 + 两处预览渲染。
<code>scripts/make_python_manual.py</code>	295	0	9	生成《Python 语法手册 (配 <code>ServerTimeDemo</code> 项目实例)》PDF。
<code>scripts/test_detail_dialog.py</code>	288	0	7	资源详情弹窗回归测试。
<code>app_version.py</code>	285	0	3	版本号与更新日志的单一数据源。
<code>scripts/make_app_icon.py</code>	285	0	10	生成应用图标 (<code>app.ico</code>), 并同步更新内嵌 base64 副本。
<code>scripts/test_expiry_table.py</code>	277	0	4	到期表格「单元格收敛」回归测试。
<code>study_demo_db.py</code>	271	0	20	Python 学习模块数据库层

模块	行数	类	函数	职责
backend_page.py	244	1	0	后台首页页面类。
scripts/check_name_scope.py	244	0	7	名字与作用域检查 —— 只读分析，不修改任何被检查文件。
scripts/test_shell_ui.py	242	0	3	外壳回归测试：自绘侧栏 + 顶栏 + 主题 token 的运行验证。
credential_process_dialogs.py	233	3	0	账号中心与流程中心相关弹窗。
scripts/scan_source_writers.py	215	0	3	源码写入扫描 —— 找出会「就地重写源码」的脚本（只读分析，绝不修改文件）。
scripts/runtime_smoke.py	214	0	2	运行时冒烟测试 —— 真实执行关键代码路径，不依赖静态推断。
embedded_admin_tools/login_memory_service.py	212	2	1	(无模块 docstring)
scripts/test_release_meta.py	210	0	7	发布元信息回归：版本号 / 更新日志 / CHANGELOG.md 是否一致。
scripts/test_log_setup.py	203	0	2	log_setup 回归测试 —— 真实写文件、真实读回，逐条断言。
credentials_page.py	195	1	0	账号中心页面类。
expiry_page.py	185	1	5	到期管理页面类。
log_setup.py	176	0	5	全局日志配置 —— 在程序入口调用一次，全项目模块自动生效。
page_components.py	176	0	10	页面级组件层。
dialog_form_style.py	167	0	7	弹窗表单统一样式。
process_page.py	152	1	0	流程中心页面类。
scripts/check_exe.py	147	0	2	打包产物体检：exe 时间戳 / 体积 + 从归档里反向核对源码真的进去了。
scripts/window_shot.py	139	1	3	把 Windows 窗口抓成 PNG —— 用 PrintWindow，不依赖桌面是否可见。
scripts/dep_inventory.py	137	0	2	依赖清单盘点 —— 静态提取全项目 import，分类为 标准库 / 第三方 / 项目内。
scripts/run_all_tests.py	126	0	2	一条命令跑完全部回归套件，并汇总成一张表。
scripts/verify_icon_generators.py	113	0	3	图标生成器等价性验证 —— 只读比对，不写入任何目标文件。
scripts/gen_app_icon_service.py	69	0	2	由 app.ico 生成内嵌图标的服务模块。
scripts/gen_app_icon.py	61	0	2	生成主程序图标 app.ico（简化版占位图）。

模块	行数	类	函数	职责
scripts/gen_icon_service.py	61	0	1	由 base64 文本生成内嵌图标的服务模块。
embedded_admin_tools/api_client.py	55	1	0	(无模块 docstring)
scripts/gen_changelog.py	50	0	2	从 app_version.VERSION_HISTORY 生成仓库根目录的 CHANGELOG.md。
embedded_admin_tools/crypto_service.py	22	1	0	(无模块 docstring)
embedded_admin_tools/error_code_service.py	7	0	0	兼容包装：统一转发到 embedded_admin_tools.services.error_code_service。
embedded_admin_tools/interface_service.py	7	0	0	兼容包装：统一转发到 embedded_admin_tools.services.interface_service。

B.1 数据目录结构

1	ExpiryManager_Data/	← 全部运行期数据都在这一个目录里
2	—— expiry_manager.db	主数据库（24 张表）
3	—— login_memory.json	登录记忆（含密码密文）
4	—— logs/app.log	轮转日志（1 MB × 3 份）
5	—— account_images/	账号中心上传的图片
6	—— study_notes_images/	笔记图片
7	—— study_notes_attachments/	笔记附件
8	—— process_flow_images/	流程图图片
9	—— ...	导出文件、历史记录等

▲ 目录名与迁移映射见 main.py 的 _MIGRATION_MAP

所有数据放一个目录，是给「备份」用的

一个程序的数据散落在十几个地方时，「备份」就变成了需要文档的操作。收进一个目录之后，用户只需要「拷走这个文件夹」。

配套的设计是 `_MIGRATION_MAP + .migrated` 标记文件：首次启动时把散落在程序目录里的老文件搬进 `ExpiryManager_Data/` 并留下标记，之后不再执行。迁移只做一次、且可重复检测，这也是幂等思想的一个应用。

B.2 测试套件与基线

项目共 11 个回归套件，下表是本次生成时的实测结果（跑一遍现统计）：

套件	通过	失败
test_notes_markdown	60	0
test_notes_editor	55	0
test_shell_ui	35	0
test_tools_launcher	230	0

套件	通过	失败
test_log_setup	17	0
test_expiry_table	23	0
test_detail_dialog	35	0
test_release_meta	82	0
test_todo	247	0
test_todo_ui	251	0
runtime_smoke	18	0
合计	1053	0

附录 C · 坑对照表

这份附录把前面各处散落的「不要这么写」集中成一张表，方便当检查清单用。左边是**错误写法**，中间是**症状**，右边是**正确写法**。

C.1 语法与语义类

错误写法	症状	正确写法
<code>def f(x, items=[])</code>	所有调用共享同一个列表，越追加越长	<code>items=None + 函数内 if items is None: items = []</code>
<code>@dataclass</code> 里 <code>tags: list = []</code>	同上，且更难发现	<code>field(default_factory=list)</code>
<code>for x in items: items.remove(x)</code>	元素被跳过，删不干净	遍历 <code>list(items)</code> 副本，或重建列表
<code>for k in d: del d[k]</code>	<code>RuntimeError: dictionary changed size</code>	先 <code>for k in list(d):</code>
循环里 <code>lambda: f(i)</code>	所有回调都用最后的 <code>i</code>	<code>lambda i=i: f(i)</code> ，或用工厂方法
<code>x == None</code>	自定义 <code>__eq__</code> 时结果可能不符预期	<code>x is None</code>
<code>if not x:</code> 判数字	<code>0</code> 被当成「没有值」	<code>if x is None:</code>
<code>except:</code>	连 <code>Ctrl+C</code> 都吞掉，程序无法中断	<code>except Exception:</code>
<code>assert</code> 做输入校验	<code>python -O</code> 时校验整段消失	<code>if not ok: raise ValueError(...)</code>
<code>except Exception: pass</code>	功能长期不工作而无人知晓	至少记一条日志；或注明降级后的状态
<code>d["k"]</code> 读可能没有的键	<code>KeyError</code>	<code>d.get("k", 默认值)</code>
<code>int("")</code>	<code>ValueError</code>	<code>int(x or 0)</code> ，或包 <code>try</code>
<code>0.1 + 0.2 == 0.3</code>	<code>False</code>	<code>abs(a - b) < 1e-9</code> ；金额用 <code>int</code> 或 <code>Decimal</code>
<code>round(2.675, 2)</code>	得 <code>2.67</code>	<code>Decimal.quantize(..., ROUND_HALF_UP)</code>
<code>strptime</code> 解析 <code>9:05</code>	部分实现解析不出非零填充小时	手工 <code>int()</code> 拆（见 <code>parse_moment</code> ）
<code>strftime("%a")</code> 判星期	中文系统上输出中文	<code>date.weekday()</code> 数值判断

C.2 文件与编码类

错误写法	症状	正确写法
<code>Path.write_text(...)</code> 改源码	整个文件的行尾被翻成 <code>CRLF</code> ，diff 爆炸	<code>read_bytes + write_bytes</code>

错误写法	症状	正确写法
<code>open(p, "w")</code> 不写编码	中文在 Windows 上乱码或报错	<code>encoding="utf-8"</code>
CSV 导出用 <code>open(p, "w")</code>	每行多一个空行	<code>newline=""</code> (+ <code>utf-8-sig</code> 给 Excel)
程序内部读写用 <code>utf-8-sig</code>	JSON 解析失败 (BOM)	内部一律 <code>utf-8</code>
<code>__file__</code> 定位数据目录	打包后数据写进临时目录, 关掉就丢	<code>sys.frozen + sys.executable</code>
一次替换多个锚点不校验	锚点写错时静默改错地方	每处锚点 断言恰好命中一次 , 否则整体拒写
补丁脚本用 <code>findall</code> 取多分组	拿到的是元组列表, <code>hits[0]</code> 不是字符串	用 <code>finditer</code> 并按 <code>group(n)</code> 取
脚本里写转义引号	反斜杠被链路吃掉, 外层字符串提前闭合	内容一律用单引号 / 三单引号
全量替换不做行数复核	行尾被改动而看不出来	<code>git diff --numstat</code> 核对变更行数

C.3 GUI 类

错误写法	症状	正确写法
<code>command=self.save()</code>	绑定时就执行一次, 回调变成 <code>None</code>	<code>command=self.save</code>
<code>def handler(self):</code> 配 <code>bind</code>	<code>TypeError</code> : 少一个位置参数	<code>def handler(self, event=None):</code>
先 <code>pack expand=True</code> 的控件	后面 <code>pack</code> 的控件被饿死, 完全不显示	先 <code>pack</code> 固定尺寸控件, 最后 <code>pack</code> 内容区
滚动条后于滚动区 <code>pack</code>	滚动条被挤出可视区	滚动条先 <code>pack</code>
子类 <code>__init__</code> 里先干别的	<code>TclError</code> : <code>self</code> 还不是控件	第一句 <code>super().__init__(...)</code>
主窗口 <code>withdraw()</code> 后查 <code>ismapped()</code>	所有控件都返回 <code>0</code> , 误判成「被饿死」	先 <code>deiconify()</code> 再判定
布局前查 <code>winfo_width()</code>	得到 <code>1</code> (还没算)	先 <code>update_idletasks()</code> / <code>update()</code>
缓存 Tk 对象不探活	解释器换过后 <code>TclError: invalid command name</code>	用前调一个无害方法探活, 失败就清缓存
销毁后继续用控件	同上	<code>if w.winfo_exists(): w.destroy()</code>
回调里不 <code>try</code>	异常被 Tk 事件循环吞掉, 界面静默失灵	回调内部 <code>try/except</code> + 记日志
用线程更新界面	随机崩溃, 极难复现	用 <code>after()</code> 回到主线程
定时器只在成功路径续期	一次异常后定时器永久死掉	<code>finally</code> : 里续期
退出时不取消定时器	回调打到正在销毁的控件上	置退出标志, 回调先检查它

C.4 数据与打包类

错误写法	症状	正确写法
f-string 拼 SQL	SQL 注入；名字带单引号直接语法错误	? 占位符 + 参数元组
循环里逐条 commit	慢（每条一次磁盘刷写）	executemany + 只 commit 一次
不设 row_factory	只能按下标取值，加一列就让取值全错位	conn.row_factory = sqlite3.Row
不设 PRAGMA foreign_keys	外键形同虚设	每个连接显式 PRAGMA foreign_keys = ON
ALTER TABLE 前不查列	重复启动报「duplicate column」	先 PRAGMA table_info 再按需补
新列不给默认值	老数据行没有合法值	ADD COLUMN ... NOT NULL DEFAULT ''
只按键排序	顺序随内部存储变化，看起来会「自己变」	加第二级 id ASC
改结构时删列	老用户数据丢失	加列永远只做加法
打包脚本先清空 dist	用户运行期数据永久丢失	只删制品，数据另存另拷
只信打包日志	漏模块只在运行版炸	打开归档 反向核对 符号
pip freeze 覆盖依赖锁	把间接依赖锁成当前版本，难以复现	只锁直接依赖

附录 D · 工具脚本与开发流程

项目跑到后来长出了一套自己的开发工具。它们既是「本项目怎么维护」的说明，也是写 Python 工具脚本的现成参考。

脚本	作用	值得学的地方
<code>scripts/run_all_tests.py</code>	一条命令跑完全部套件并汇总成表	开场先探 <code>tkinter</code> 是否可用，解释器不对时 直接说该换哪个 ，不闷头跑出一片假红
<code>scripts/window_shot.py</code>	把窗口抓成 PNG	<code>PrintWindow + PW_RENDERFULLCONTENT</code> ；抓完校验颜色数， 只抓到一种就抛错
<code>scripts/check_exe.py</code>	打包后反向核对归档里的符号	递归遍历 <code>code object</code> 的 <code>co_names / co_consts</code>
<code>scripts/test_*.py</code> (11 个)	回归测试	每个都用 <code>check(名称, 条件, 现场值)</code> 的写法，失败时打印真实值
<code>scripts/gen_changelog.py</code>	从 <code>app_version.py</code> 生成 CHANGELOG	单一数据源 ：版本号只有一处，日志自动重建
<code>project_audit.py</code>	AST 静态审计	函数长度、嵌套深度、重复代码
<code>check_name_scope.py</code>	补 <code>pyflakes</code> 盲区	<code>except</code> 名字作用域、闭包作用域
<code>prune_unused_imports.py</code>	批量删未使用导入	字节级改写 + 语法安全校验
<code>dep_inventory.py</code>	依赖清单	统计模块级 <code>import</code> 的第三方包

D.1 本项目自己的「补丁纪律」

前面 13.1 与 13.2 讲的是行尾，这里把它拼成一套完整的操作流程——这是本项目这几轮迭代里最核心的一条工程经验。

- 写一个补丁脚本，而不是手工改文件。**脚本里每处替换都带断言：锚点必须**恰好命中一次**，否则整体拒写。这一步拦住的是「替换到了别的地方」和「文件已经被改过」。
- 顺序化执行，不要在同一个文件上并行编辑。**同一文件的两处修改若并行提交，后一次的写入会基于旧内容，**静默丢掉前一次的改动**。
- 字节读、字节写** (`read_bytes / write_bytes`)，中途 `decode` 处理文本、`encode` 写回，全程不碰文本模式的行尾翻译。
- 先记下当前的行尾指纹**：纯 CRLF 的文件，断言 `count(b"\r\n") == count(b"\n")`；纯 LF 的文件，断言 `count(b"\r\n") == 0`；混排的文件 (`main.py` 有 167 个单独 LF)，断言这个数量没变。
- 写完立刻** `py_compile.compile(path, doraise=True)`，语法错误当场暴露。
- 收尾用** `git diff --numstat` **核对**：变更行数与预期不符，说明动到了不该动的地方，`git checkout -- .` 回退重来。

这套流程里最容易被跳过、也最危险的一条

「删除」最危险，仅次于它的就是「全量替换」。

本项目有一条硬规则：**不用** `git rm <目录>/<文件>`。在某个 Git 版本下，只要路径里带 `/`，`git rm` 会删掉整个顶层目录树——曾经因此一次性误删 25 个文件。正确做法是 `rm <file>` 再用 `git add -A` 让 Git 记录，而且**删除之前先提交**，这样任何误删都能 `git checkout --` 完整恢复。

D.2 这套流程的通用形状

把上面六步抽象出来，它与具体的项目、语言都无关：

任何「程序化改写文件」都适用的四条

- ① **先验证前提**（锚点唯一、现状符合预期），再动手；
- ② **宁可整体失败，不要部分成功**（一处不对劲就什么都不写）；
- ③ **写完立刻做一次独立校验**（编译、行数、差异）；
- ④ **保证可回退**（先提交，再改）。

这四条合起来就是一个最小可用的「安全批量改写」协议。

D.3 待办（项目的下一步）

以下几条写在项目自己的记忆文件里，属于已经排定但还没做的：

- **拆** `main.py`（当前 5,400 余行）—— 入口、主窗口、通用工具函数三者分离。
- `todo_db.py` 的**迁移逻辑表驱动化** + SQL 标识符白名单。
- `Database` 的 **Mixin 拆分**—— 现在 24 张表的操作都在一个类里。
- **引入 ruff 与 mypy**，把静态检查从「只看告警数」推进到「有规范标准」。
- **其余页面的空状态**—— 待办模块的空状态已经做完，其它页面还没有。

结语

这本手册从头到尾只做了一件事：**把语法点钉在真实代码上**。如果你读完之后记住的不是「Python 有推导式和装饰器」，而是「定时器续期要放 `finally`」「改源码要走字节」「`field(default_factory=list)` 是为什么」——那它就起作用了。

附录里的模块清单、体量统计、测试基线都是**生成时现算的**，所以只要重新跑一次生成脚本，它们永远与仓库同步；而正文里的代码是手抄的，改动后需要人工同步。这一点也提醒了一件更普适的事：**能自动化的东西就让机器算，手工抄写的内容一定要标注它的保质期**。